

Killer Sudoku

Architecture Documentation

nach arc42 v8.2

Project	Killer Sudoku (Variant: Cage Sums)
Context	Skills Battle 2026 — Application Development
Author	Tim Fischer
Status	Final
Date	2026-05-27

Inhaltsverzeichnis

Architektur-Dokumentation — Killer Sudoku · Skills Battle 2026 Standard: arc42 v8.2 · 12 Kapitel · Status: Final Author: Tim Fischer · Stand: 2026-05-27

Kapitel

#	Kapitel
1	Einführung und Ziele
2	Randbedingungen
3	Kontextabgrenzung
4	Lösungsstrategie
5	Bausteinsicht
6	Laufzeitsicht
7	Verteilungssicht
8	Querschnittliche Konzepte
9	Architekturentscheidungen
10	Qualitätsanforderungen
11	Risiken und Technische Schulden
12	Glossar

Konventionen in diesem Dokument

Konvention	Bedeutung
Querverweise	"Kapitel X" verlinkt auf den entsprechenden Abschnitt in diesem Dokument
Mermaid-Diagramme	als SVG eingebettet (Klassen-, Sequenz-, ER-, Flow-Diagramme)
Zitate	wörtliche Auszüge aus der Aufgabenstellung sind in Anführungszeichen gesetzt
ADR-Nummern	Kapitel 9 nummeriert Entscheidungen als ADR-001 bis ADR-014
Identifizier	UCxx = Use Case · Vxx = Validation Rule · ACxx.y = Acceptance Criterion · Txxx = Test Case

1 Einführung und Ziele

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

§1.1 Aufgabenstellung

Die Anwendung implementiert eine **Killer-Sudoku-Variante** als Web-Applikation. Anstelle des klassischen Sudoku werden zusätzlich **Cage-Summen** (Summen über mehrere Zellen) als Constraints vorgegeben. Die App muss Plan, Implementation und Tests gemäss den im Wettbewerbs-README spezifizierten Use Cases liefern.

Spielregeln (wörtlich aus README §1)

Die Lösung eines Killer-Sudoku-Puzzles muss vier Bedingungen erfüllen (README §1, "the following conditions are met"):

"Each row, column, and nonet contains each number **exactly once**." "The sum of all numbers in a cage **must match** the small number printed in its corner." "**No** number appears more than once in a cage." "The solution **must be unique**."

Diese vier Regeln sind in der Anwendung als Validierungslogik abzubilden (siehe **V09**, **V10**) und werden in Use Case **UC09** sowie im Solver **UC11** geprüft.

Funktionaler Scope

Die Anwendung deckt die im README §2.1 vorgegebenen Use Cases **UC01–UC11** sowie drei selbst gewählte zusätzliche Use Cases **UC12–UC14** ab (gemäss README §2.3, "Think of 3 additional use cases"):

- **UC01–UC11** — Read Rules, Register, Login, Enter Puzzle, Save Puzzle (Solvability-Check), Solve Puzzle, Hint, Highscore, Check Solution, Save Result, Auto Solve
- **UC12** — Browse / Filter Puzzles
- **UC13** — Pause & Resume Game
- **UC14** — Pencil Marks

Eine vollständige Übersicht inkl. Acceptance Criteria findet sich in **Use-Cases-Dokument**. Die Service-/Screen-/DB-Zuordnung ist in **Funktionalitäts-Matrix** tabelliert.

Design-Freiheit (README §1.1)

Das README erlaubt explizite Designfreiheit: "You are free to design the application. Your task is to create mockups." Die Mockups sind in **Mockup-Briefings** als Figma-Briefings dokumentiert (Screens S1–S7 + Layout).

§1.2 Qualitätsziele

Die folgenden drei Top-Qualitätsziele leiten alle Architektur- und Implementierungsentscheidungen. Sie sind so gewählt, dass sie aus den **strikten Vorgaben des README** ableitbar und über Tests messbar sind.

#	Qualitätsziel	Begründung (README-Bezug)	Messbarkeit
Q1	Korrektheit des Solvers (Funktionale Korrektheit)	README §1 fordert "The solution must be unique. " und §2.1 UC4 fordert "Solutions must be calculated with an algorithm. " Der Solver ist die zentrale Logik-Komponente — falsche Ergebnisse zerstören UC05, UC07, UC09.	Solver-Unit-Tests gegen die 2 README-Beispiele + Edge-Cases (unlösbar, mehrdeutig). Performance-Boundary < 2s (siehe AC11.2).
Q2	Lösungs-Eindeutigkeit (Konsistenz beim Speichern)	README §1 "The solution must be unique." + §2.1 UC5 "The puzzle can only be saved if it is solvable. " Multi-Solution-Puzzles werden abgelehnt — nicht nur Zero-Solution (siehe V07).	Integration-Test: Speichern eines mehrdeutigen Puzzles → kein DB-Row.
Q3	Test-Abdeckung & Test-Plan-Termin	README §3.1 fordert pro UC mind. 1 positiver + 1 negativer + (wo möglich) 1 Boundary-Test. README §1.4: "submission of the planned test cases must take place by 12 o'clock. "	Test-Plan vor 12:00 abgegeben; Unit-Tests in Test-Framework lt. README §3.2 (" should be implemented with a test framework "). Coverage-Ziel siehe Kapitel 10 .

Weitere relevante Qualitätsmerkmale (sekundär)

- **Sicherheit** — README §2.1 UC2: "a login **is required.**" → Authentifizierung, geschützte Routen, sichere Passwort-Speicherung (siehe **V03**, **V16**).
- **Bedienbarkeit** — README §1.1 fordert "appropriate navigation" + "controls for all functions" → vollständig adressiert in **Mockup-Briefings**.
- **Wartbarkeit** — Klare Trennung Service-Layer / DB-Layer / UI (siehe Service-Interfaces in **Funktionalitäts-Matrix**).

§1.3 Stakeholder

Rolle	Erwartung	Berührungspunkte
Wettbewerbs-Prüfer (Skills Battle Jury)	Konforme Submission gemäss README §1.4 — ZIP-File, Doku als PDF, Source-Code, Executables, DB-Script. Bewertung gegen UCs und Test-Protokoll.	<code>AppDev_Name_FirstName.zip</code> (siehe Kapitel 2 §2.2).
Wettbewerber (Tim Fischer, Implementierer)	Zeit-effiziente, testbare Architektur. Strikte Vermeidung von Scope-Creep. Reproduzierbares lokales Setup.	Source-Code, Test-Suite, lokaler Build mit <code>dotnet</code> .
End-Nutzer (Spieler der App)	Funktionierender Killer-Sudoku-Spielfluss: Account → Puzzle wählen → spielen → Hint → Lösung prüfen → Highscore. UI-Sprache deutsch.	UI-Screens S1–S7 (Mockup-Briefings).
Puzzle-Ersteller (User-Subgruppe, kreiert Puzzles via UC04/UC05)	Verlässliche Solvability-Prüfung, kein Datenverlust bei Reject.	Screen S5 — EnterPuzzle (Mockup-Briefings).

Nicht-Stakeholder (bewusst ausgegrenzt)

Folgende Rollen sind im Scope **nicht** vorgesehen (Begründung: nicht in README spezifiziert, keine UC-Anker):

- Admin-Rolle / Moderation
- Externe Systeme (Social-Login, Payment, Analytics)
- Mehrspieler-/Live-Funktionen

Verweise

- Use Cases UC01–UC14
- Validation Rules V01–V16
- **Funktionalitäts-Matrix**
- [Kapitel 2 — Randbedingungen](#)
- [Kapitel 3 — Kontextabgrenzung](#)

2 Randbedingungen

arc42 v8.2 · Killer Sudoku App Quelle für Pflicht-Vorgaben: **Aufgabenstellung**

Randbedingungen sind **nicht-verhandelbare** Vorgaben, die das Design und die Implementierung einschränken. Sie sind in technische, organisatorische und Konventions-Constraints gegliedert.

§2.1 Technische Randbedingungen

Plattform & Stack

Constraint	Wert	Quelle / Begründung
Runtime / Framework	.NET 10	Wettbewerbs-Stack-Festlegung Tim Fischer. Dokumentiert in Use-Cases-Dokument (Stack-Header).
UI-Technologie	Blazor Server	Server-Side Rendering mit SignalR-Circuit; spec-konform für interaktive Web-Anwendung lt. README §1.1 (Designfreiheit).
Datenbank	MS-SQL Server Express	README §1.3 erlaubt "MySQL or MS-SQL server running on your machine." Entscheidung: MS-SQL Express. DB-Name strikt: <code>sudoku</code> (README §1.3: "create a database named 'sudoku'").
ORM / Data-Access	Entity Framework Core 10	Entschieden (siehe ADR-001). <code>SudokuDbContext</code> + EF-Migrations; das Submission-Skript Datenbank-Skript wird per <code>dotnet ef migrations script</code> aus den Migrations exportiert.
Authentication	ASP.NET Core Identity (Cookie-basiert)	Aus README §2.1 UC2 ("a login is required") abgeleitet; siehe V03 + V16 .

Test-Frameworks

README §3.2 (wörtlich): "Running the test cases **should be implemented with a test framework** and is part of the submission (e.g. JUnit in Java)." Für .NET ergibt sich:

Test-Typ	Framework	Zweck
Unit-Tests	xUnit	Service-Layer, Solver, Validation-Logic (Q1, Q3 — siehe §1.2).
Component-Tests	bUnit	Blazor-Komponenten (Grid, CageEditor, Toolbar).
E2E / UI-Tests	Playwright (.NET)	Smoke-Tests kritischer Flows. README §3.2 erlaubt "UI tests can be run without a test framework" — Playwright ist optionale Erweiterung.

Datenbank-Setup (README §1.3, wörtlich zitiert)

- **Database-Name:** "sudoku" (Pflicht-Name aus README §1.3)
- **Schema-Freiheit:** "You are free to build the database schema."
- **SQL-Script-Pflicht:** "Create for each table a SQL statement and save them in a script called **Datenbank-Skript**."
- **Constraints-Pflicht:** "Don't forget to create the primary and foreign key constraints."

Das Schema und die Constraints sind in **ER-Modell** modelliert; das SQL-Script wird in `db/sudoku.sql` ausgeliefert.

Browser-Support

Browser	Status	Begründung
Chrome (current stable)	Pflicht-Target	E2E-Tests laufen primär in Chromium-Engine.
Firefox (current stable)	Sekundär-Target	Manuelle Smoke-Verifikation.
Safari / Edge	Best-effort	Nicht im README spezifiziert, keine Test-Verpflichtung.

Entwicklungsumgebung (IDE-Auswahl)

IDE	Status
Visual Studio 2022/2026	Primär (Blazor-Server-Templates, MS-SQL-Tools)
Visual Studio Code + C# Dev Kit	Alternative
JetBrains Rider	Alternative

Die Wahl ist intern; das Build- und Test-Setup muss in **allen drei** funktionieren (kein IDE-gebundenes Projekt-Setup).

UNCLEAR: Eine exakte minimale .NET-SDK-Version wird vom README nicht vorgegeben.
Annahme: .NET 10 (LTS-Kandidat zum Wettbewerbszeitpunkt).

§2.2 Organisatorische Randbedingungen

Submission-Termine (README §1.4 + §3.1)

Termin	Pflicht (wörtliches Zitat)
12:00 Uhr (Wettbewerbstag)	"submission of the planned test cases must take place by 12 o'clock " (README §1.4 + §3.1).
Ende Wettbewerbstag	Komplette Submission (ZIP) gemäss README §1.4.

Deliverables (README §1.4, wörtlich)

"All deliverables must be submitted in a zip file named `AppDev_Name_FirstName.zip` ." "Only the contents of the zip file will be considered for the evaluation."

Pflicht-Inhalte der ZIP-Datei:

Deliverable	Format	README-Zitat
Dokumentation inkl. Test-Protokoll	PDF	§2.6: "For the delivery, create a PDF document ."
Executable Files	Build-Artefakte (.exe / <code>dotnet publish</code> Output) + Container-Image-Tag (<code>ghcr.io/tim-fischer-zh/killer-sudoku:latest</code>)	§1.4: "executable files"
Source-Code	Vollständig, inkl. <code>Dockerfile</code> , <code>docker-entrypoint.sh</code> , <code>docker-compose.yml</code> , <code>.github/workflows/docker.yml</code>	§1.4: "the source code"
DB-Script	Datenbank-Skript (im Container automatisch beim ersten Start applied)	§1.3: "save them in a script called <code>sudoku.sql</code> "

Container-Variante (zusätzlich zur Native-Variante)

Die Submission liefert ein **Single-Image-Container-Setup** als bevorzugten Startpfad (siehe [ADR-015](#) und [§7.5](#)). Der Prüfer kann zwischen drei Pfaden wählen:

1. `docker pull` + `docker run` — Image direkt von `ghcr.io` (≈ 1.5 GB), App in unter 60 Sek startklar.
2. `docker compose up --build` — lokal aus dem Repo-Root bauen.
3. **Native** — MS-SQL Express + .NET 10 SDK lokal, **Datenbank-Skript** manuell deployen (Standard-Pfad, vollständig in [§7.2](#) / [§7.3](#) dokumentiert).

README §1.3 ("running on your machine") wird in allen drei Pfaden erfüllt — Container laufen auf der Maschine des Prüfers.

Dokumentations-Inhalt (README §2.6, wörtlich)

Die Dokumentation muss folgende Sektionen enthalten:

"Mockup · Database diagram · Class diagram · Additionally chosen use cases · Validation rules · Test protocol"

Diese Punkte werden über die arc42-Kapitel und die in `docs/` referenzierten Spezialdokumente abgedeckt:

README-Sektion	Quelle im Repo
Mockup	Mockup-Briefings + <code>docs/mockups/*.png</code>
Database diagram	ER-Modell
Class diagram	Kapitel 5 — Bausteinsicht (TBD)
Additionally chosen use cases	Use-Cases-Dokument (UC12–UC14)
Validation rules	Validation-Regeln
Test protocol	Test-Protokoll (TBD)

Bewertungsmaterial-Beschränkung

README §1.4: "Only the contents of the zip file will be considered for the evaluation."

Konsequenz: Externe Repos, Cloud-Builds oder Online-Demos werden **nicht** in die Bewertung einfließen. Alles, was zur Bewertung beitragen soll, muss im ZIP enthalten sein.

§2.3 Konventionen

Sprach-Konvention

Bereich	Sprache	Begründung
UI-Texte (Buttons, Labels, Fehlermeldungen)	Deutsch	Implementierer + Zielgruppe in DACH-Region; konsistent mit Mockup-Briefings (Mockup-Briefings) und Validation-Texten (Validation-Regeln).
Code-Identifizier (Klassen, Methoden, Variablen)	Englisch	.NET-Konvention; siehe Service-Interfaces in Funktionalitäts-Matrix .
Architektur-Doku (arc42)	Deutsch	arc42-Standard im DACH-Kontext; README-Zitate bleiben original-englisch in "...".
DB-Schema-Namen	Englisch	Englische Tabellennamen (<code>AppUser</code> , <code>Puzzle</code> , <code>Cage</code> , ...); siehe ER-Modell .

Code-Style

- **C# Style:** Microsoft .NET Coding Conventions (PascalCase für Public, camelCase für lokal, `_camelCase` für private fields).
- **Nullable Reference Types:** aktiviert (`<Nullable>enable</Nullable>` im csproj).
- **Async-Konvention:** I/O-bound Methoden mit `Async` -Suffix und `Task` -Return — sichtbar an Service-Interfaces (z.B. `IPuzzleService.SaveIfSolvableAsync` , siehe **Funktionalitäts-Matrix**).
- **Immutability bevorzugt** wo idiomatisch (DTOs als `record`).

Sicherheits-Konventionen (aus Validation-Regeln abgeleitet)

Konvention	Quelle
Login required für alle geschützten Routen (Puzzles, Spielen, Highscore)	README §2.1 UC2 + V16
Passwort wird NIE im Klartext gespeichert	V03
Generische Login-Fehlermeldung (kein User-Existenz-Hinweis)	V03 + AC03.1
CSRF-Token auf allen POST/Form-Submissions (Blazor-Standard)	V15
HTML-Encoding auf User-Content (<code>@</code> -Razor-Syntax)	V14
Defense in Depth: kritische Validierungen auf Client + Server + DB-Constraint	Header Validation-Regeln

Verweise

- [Kapitel 1 — Einführung und Ziele](#)

- [Kapitel 3 — Kontextabgrenzung](#)
- Validation Rules
- **ER-Modell**
- **Funktionalitäts-Matrix**

3 Kontextabgrenzung

arc42 v8.2 · Killer Sudoku App Quelle (autoritativ): **Aufgabenstellung**

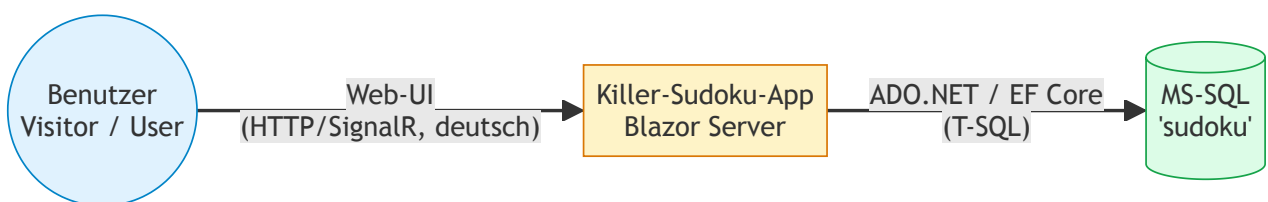
Die Kontextabgrenzung beschreibt die Systemgrenze: **welche Akteure** mit der Anwendung interagieren, **welche Nachbarsysteme** angesprochen werden, und über **welche Schnittstellen** die Kommunikation läuft. Sie unterscheidet fachlich (welche Akteure / welche Use Cases) und technisch (welche Protokolle / Komponenten).

§3.1 Fachlicher Kontext

System-Grenze

Die Killer-Sudoku-App ist primär ein **lokal betriebenes System** mit einer Web-UI (Spec-konform laut README §1.3). Es existieren **keine externen Fachsysteme** (kein Drittanbieter-Login, keine externen APIs, keine Bezahl-/Mailservices).

Zusätzlich: Eine **Bonus-Production-Instanz** der App ist unter <https://web17skill.com> erreichbar (siehe §7.6 [Production-Deployment](#) und [ADR-016](#)). Diese Variante ist **nicht** Pflicht laut Aufgabenstellung und ändert weder den fachlichen Kontext noch die Systemgrenze — sie ist nur eine andere Betriebsumgebung desselben Images.



Akteure

Akteur	Beschreibung	Berührungspunkte (UC)
Visitor	Nicht angemeldeter Browser-Nutzer. Darf nur die öffentlichen Seiten sehen.	UC01 (Rules), UC02 (Register), UC03 (Login)
User	Angemeldeter Spieler. Darf Puzzles erstellen, spielen, Highscore sehen.	UC04–UC10 , UC12–UC14
System (intern, kein externer Akteur)	Triggert automatische Folge-UCs (z.B. UC10 nach erfolgreichem UC09).	UC10 , UC11 (intern aufgerufen von UC05/UC07/UC09)

Keine externen Akteure / Nachbar-Systeme

Folgendes ist **bewusst nicht** Teil des Kontexts (siehe [Kapitel 2](#)):

- Kein Identity-Provider (Google/Microsoft/Apple-SSO)
- Kein Email-Service (kein Email-Verifikations- oder Password-Reset-Flow im Scope)
- Keine externen Highscore-/Leaderboard-APIs
- Kein Cloud-Backend — Anwendung läuft lokal mit lokaler DB

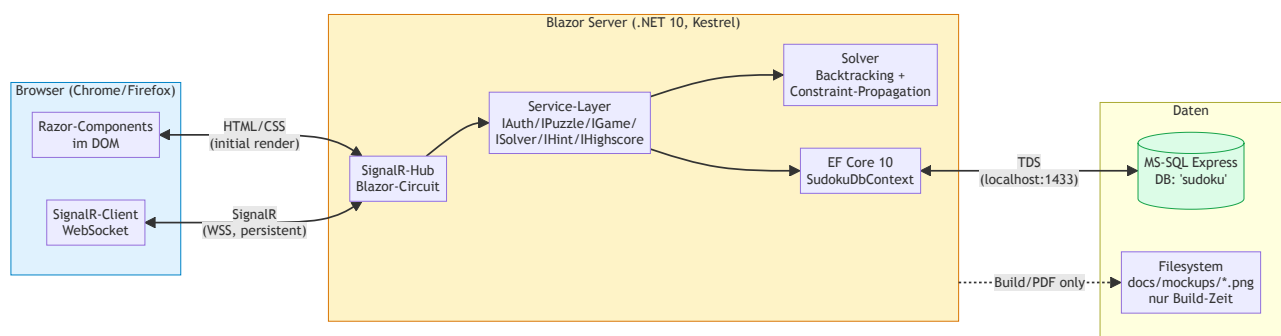
README §1.3 (wörtlich): "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine.**" → klare Lokal-Vorgabe.

Eingaben / Ausgaben über die System-Grenze

Richtung	Daten	Kanal
User → App	Login-Credentials, Puzzle-Definition (Cages + Summen), Zelleneingaben (1–9), Hint-/Check-/Pause-Aktionen	HTTP-POST (initial) + SignalR-WebSocket (Blazor-Server-Circuit)
App → User	Gerendertes 9×9-Grid, Cage-Borders + Summen, Timer, Hint-Ergebnis, Lösungs-Validierungs-Status, Highscore-Liste, Fehlermeldungen (deutsch)	HTML/CSS (initial) + SignalR-Patches (UI-Updates)
App → DB	INSERT/UPDATE/SELECT auf <code>AppUser</code> , <code>Puzzle</code> , <code>Cage</code> , <code>CageCell</code> , <code>Game</code> , <code>GameCell</code> , <code>PencilMark</code> , <code>HintLog</code> (siehe ER-Modell)	EF Core über ADO.NET / TDS-Protokoll
DB → App	Abfrage-Resultate, generierte Primary Keys, View-Daten (<code>vw_Highscore</code>)	TDS

§3.2 Technischer Kontext

Komponenten & Protokolle



Schnittstellen-Details

Schnittstelle	Protokoll	Format	Notiz
Browser ↔ Server (Initial-Page-Load)	HTTPS	HTML	Erste Request liefert Razor-generiertes HTML inkl. Blazor-Bootstrapper.
Browser ↔ Server (UI-Interaktion)	SignalR über WebSocket (WSS)	binär (MessagePack)	Persistente Verbindung pro Blazor-Circuit. UI-State serverseitig.
Server ↔ DB	TDS via <code>Microsoft.Data.SqlClient</code>	T-SQL	Local-Loopback (<code>localhost:1433</code> oder Named Pipe).
Server ↔ Filesystem	nur Build-Zeit	PNG (Mockups), PDF (Doku)	Keine Runtime-File-IO ausser Logs.

Blazor-Server-Circuit (zentrales technisches Konzept)

- **State serverseitig:** Game-State, aktuelle Zell-Auswahl, Pencil-Marks-Status leben in C#-Components auf dem Server.
- **UI-Updates push-basiert:** Server pusht Diff-Patches an den Client über SignalR.
- **Skalierung:** Eine Circuit pro Browser-Tab. Bei Disconnect → State kann via Reconnect wiederhergestellt werden (Standard-Blazor-Mechanismus; siehe auch **UC13** für explizite Pause/Resume-Semantik).

Lokales Setup (Single-Machine-Deployment)

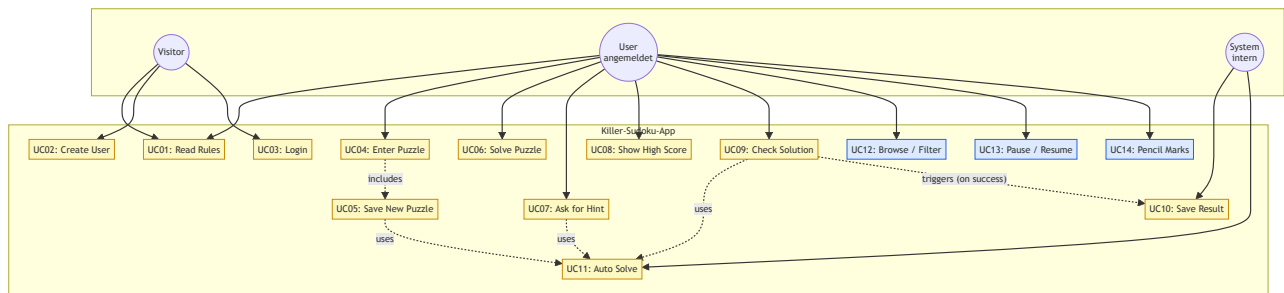
Aus README §1.3 ("running on your machine") folgt: Die App und die DB laufen **auf dem gleichen Host**. Es gibt keinen Multi-Tier-Deployment-Plan. Details siehe spätere Kapitel:

- [Kapitel 5 — Bausteinsicht](#) (TBD): innere Struktur Service-Layer
- [Kapitel 7 — Verteilungssicht](#) (TBD): lokales Deployment-Diagramm

§3.3 Use-Case-Übersicht

Vollständige UC-Spezifikation: siehe **Use-Cases-Dokument**. Diese Sektion liefert nur die Aktor-Sicht und das Diagramm als Kontext-Anker.

Use-Case-Diagramm



Legende: Gelb = README §2.1 (UC01–UC11) · Blau = selbst gewählte Zusatz-UCs §2.3 (UC12–UC14).

Kurz-Mapping (Detail in Use-Cases-Dokument)

UC	Kurzbeschreibung	Akteur	Auth
UC01	Regeln + Beispiel auf Startseite	Visitor / User	public
UC02	Account erstellen	Visitor	public
UC03	Anmelden	Visitor	public
UC04	Puzzle manuell anlegen (Cages + Summen, Difficulty 1–3)	User	required
UC05	Puzzle speichern — nur wenn solvable + eindeutig	User	required
UC06	Puzzle spielen	User	required
UC07	Hint anfordern	User	required
UC08	Highscore-Liste	User	required
UC09	Lösung prüfen (Sum-Check 405 → Vollvalidierung)	User	required
UC10	Game-Resultat speichern (Time, Hints, Score)	System	required
UC11	Solver (intern für UC05/UC07/UC09)	System	n/a
UC12	Puzzle-Liste mit Filter/Sort	User	required
UC13	Spiel pausieren und fortsetzen	User	required
UC14	Candidate-Notes pro Zelle	User	required

Use-Case-Cluster (zur Test-Planung)

Cluster	UCs	Test-Fokus
Authentifizierung	UC02, UC03	Sicherheit (V01–V04, V16)
Puzzle-Lifecycle	UC04, UC05, UC11	Solver-Korrektheit (Q1), Eindeutigkeit (Q2)
Spiel-Flow	UC06, UC07, UC09, UC10, UC13, UC14	Game-State-Konsistenz (V12)
Diskover & Stats	UC01, UC08, UC12	Read-Pfade, Sortierung/Pagination

Verweise

- [Kapitel 1 — Einführung und Ziele](#)
- [Kapitel 2 — Randbedingungen](#)
- Use Cases UC01–UC14 (Detail)
- Validation Rules
- **Funktionalitäts-Matrix**
- **ER-Modell**
- **Mockup-Briefings**

4 Lösungsstrategie

arc42 v8.2 · Kapitel 4 · Killer Sudoku Bezug: [00-README.md](#) (Index) · **Use-Cases-Dokument** · **Funktionalitäts-Matrix** · **Validation-Regeln** · **ER-Modell** · **Aufgabenstellung**

Dieses Kapitel fasst die fundamentalen Entscheidungen zusammen, die alle nachfolgenden Kapitel tragen. Die Aufgabenstellung gibt **kein** spezifisches Framework vor (siehe §1.3 "free to build the database schema", §1 "your task is to plan, implement, and test"). Damit sind Technologie-Wahl und Top-Level-Zerlegung explizit Architektur-Entscheidungen — sie sind hier dokumentiert.

4.1 Technologie-Entscheidungen

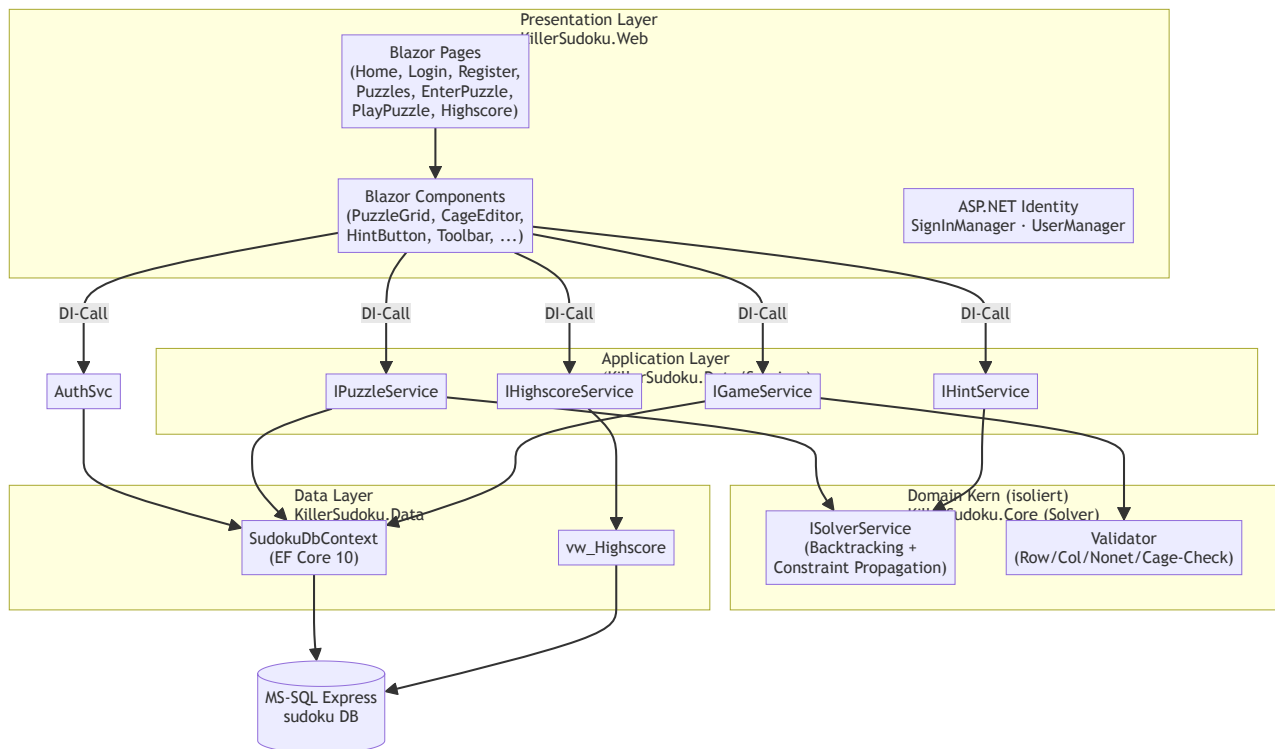
Die folgende Tabelle ordnet jede zentrale Anforderung aus dem Aufgaben-README einer technischen Lösung zu. Vollständige Begründung in [Kapitel 9 — Architekturentscheidungen](#) (TBD).

Anforderung (Quelle)	Gewählte Lösung	Begründung
"login is required" (UC02/UC03)	ASP.NET Core Identity (Cookie-Auth)	Built-in für .NET 10, PBKDF2-Hashing, CSRF/XSRF-Token frei Haus, deckt V03/V15 Defense-in-Depth
"stored in the database" (UC04)	Microsoft SQL Server Express + Entity Framework Core 10	README erlaubt explizit MS-SQL oder MySQL; MS-SQL Express ist lokal lizenzkostenfrei, native T-SQL-Constraints (CHECK/UNIQUE/Filtered Index/Trigger) tragen DB-Layer-Verteidigung aus ER-Modell
"must be calculated with an algorithm" (UC04/UC11)	Reiner C#-Solver (<code>ISolverService</code>) als isolierter Domain-Kern ohne Framework-Abhängigkeit	Pure Logik → Unit-testbar ohne DB/UI; Backtracking + Constraint-Propagation; siehe Funktionalitäts-Matrix Service-Block
"solution must be unique" (§1)	Solver bricht nach dem zweiten gefundenen Pfad ab → <code>Solutions ∈ {0,1,2}</code> (V07)	Performance-Begrenzung (AC11.2: <2 s); 2 ist Marker für "≥2"
Interaktives Grid (UC04, UC06, UC07, UC14)	Blazor Server (Razor Components + SignalR)	Server-rendered: Solver/Validierung läuft serverseitig (kein clientseitiges Cheaten möglich), <code>[Authorize]</code> -Attribut für Routen, kein zweites SPA-Frontend-Projekt nötig
"test cases ... unit tests ... test framework" (§3)	xUnit + bUnit (Component-Tests) + EF Core InMemory / Testcontainers	Standard für .NET; bUnit erlaubt Blazor-Komponenten-Tests ohne Browser; siehe Test-Protokoll
Validation-Defense (V01–V16)	3 Layer : Client (Blazor <code>EditForm</code> /Annotations) → Server (<code>IXxxService</code>) → DB (CHECK/UNIQUE/Trigger)	"Defense in Depth" aus Validation-Regeln Header — kritische Regeln auf mind. 2 Layern
Sum-Check "405" (§2.3)	Fast-Fail in <code>IGameService.CheckSolutionAsync</code> vor Voll-Algorithmus (V09)	README §2.3: "Use the value for a simple validation before checking the solution with an algorithm" — wörtlich umgesetzt

Hinweis: Die Solver-Implementierung wird über das Interface `ISolverService` aus **Funktionalitäts-Matrix** entkoppelt — der konkrete Backtracker ist eine austauschbare Detail-Entscheidung, kein Architektur-Pfeiler.

4.2 Top-Level-Zerlegung

Das System wird in eine **klassische 3-Layer-Architektur** zerlegt, ergänzt um einen **isolierten Domain-Kern** für die rechenintensive Solver-Logik. Diese Zerlegung erscheint im Detail in [Kapitel 5 — Bausteinsicht](#).



Schichten-Verantwortung:

Schicht	Verantwortung	Abhängigkeit nach
Presentation (KillerSudoku.Web)	UI-Rendering, Routing, User-Interaktion, Input-Binding, Display-State	nur Application (über Interfaces)
Application (KillerSudoku.Core Services)	Use-Case-Orchestrierung, Transaktions-Grenzen, Autorisierung, Aufruf des Solvers	Domain + Data
Domain-Kern (KillerSudoku.Core Solver)	Reine Algorithmik (Backtracking, Constraint-Propagation, Sum-Check)	keine (pure C# — kein EF, kein ASP.NET)
Data (KillerSudoku.Data)	EF-Core-Mapping, Migrations, Query-Performance, View-Zugriff	DB-Treiber

Der Solver als isolierter Domain-Kern ist die wichtigste Designtrennung: Er ist die kritische Geschäftslogik (UC11 wird von UC05, UC07, UC09 verwendet — siehe Abhängigkeits-Graph in **Funktionalitäts-Matrix** §Kritische Abhängigkeiten) und muss separat per Unit-Test verifiziert werden. Pure C# ohne Framework-Bindung erlaubt Tests in Millisekunden statt Sekunden.

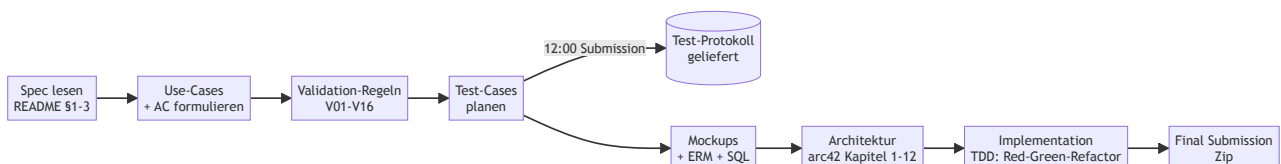
4.3 Erreichen der Qualitätsanforderungen

Die wichtigsten Qualitätsziele werden durch konkrete Patterns adressiert. Vollständige Liste in [Kapitel 10](#) (TBD); hier nur die strategie-prägenden Mappings.

Qualitätsziel	Pattern / Mechanismus	Bezug
Korrektheit — Sudoku- und Killer-Regeln werden niemals verletzt	(a) DB-Constraints als letzte Verteidigung (V05 , V06 , V08). (b) Server-seitige Re-Validierung in <code>IGameService.CheckSolutionAsync</code> (UC09). (c) Solver muss "solution must be unique" garantieren (V07).	README §1 Cage-Regeln
Performance — Solver < 2 s, UI-Reaktion < 200 ms	(a) Sum-Check 405 als Fast-Fail vor Algorithmus (AC09.1). (b) Solver bricht bei der 2. Lösung ab (AC11.2). (c) Paginierte Puzzle-Listen (AC12.2). (d) Highscore-Read derzeit als LINQ-Join im <code>HighscoreService</code> (View <code>vw_Highscore</code> existiert im Schema als Reserve, wird aktuell nicht gemappt — siehe ER-Modell).	UC11 AC11.2
Testbarkeit — 80 %+ Coverage, TDD-fähig	(a) Service-Interfaces (<code>I*Service</code>) erlauben Mock-Substitution. (b) Solver als pure Funktion testbar ohne DB. (c) bUnit für Component-Tests, EF InMemory für Integration.	Test-Protokoll
Sicherheit — kein Klartext-Passwort, kein XSS, kein CSRF	(a) ASP.NET Identity PBKDF2 (V03). (b) Razor <code>@</code> -Encoding (V14). (c) Antiforgery-Token in EditForm (V15). (d) <code>[Authorize]</code> + UserId-Ownership (V16).	UC02/UC10/UC13
Wartbarkeit — kleine, fokussierte Files	"Many small files > few large files" aus globalen Code-Standards: 200–400 Zeilen pro Komponente, Service oder Repo. Komponenten-Tree spiegelt Domain-Begriffe.	Coding-Style-Regel
Datenintegrität — keine inkonsistenten Game-States	(a) Filtered Unique Index <code>UX_Game_ActiveOnly</code> (max 1 aktives Game pro User-Puzzle). (b) Trigger <code>trg_CageCell_UniquePerPuzzle</code> (Cell-Cage-Eindeutigkeit). (c) Transaktion in <code>SaveIfSolvableAsync</code> (Puzzle + Cage + CageCell atomar).	ER-Modell Constraints

4.4 Organisatorische Strategie

Die Aufgabenstellung schreibt einen **klaren zeitlichen Ablauf** vor (§3.1: "Submission der geplanten Test-Cases bis 12 Uhr", §1.4 "Doku + Source + DB-Script"). Das diktiert die Reihenfolge der Artefakte:



Tragende organisatorische Prinzipien:

1. **Spec → Tests → Code** (TDD): Jeder Test-Case in **Test-Protokoll** verlinkt auf eine `AC*`-ID; jede AC referenziert wörtlich den README-Text via "Source"-Zeile.
2. **Solver zuerst:** Da `ISolverService` Pflicht-Dependency für UC05, UC07, UC09 ist (siehe **Funktionalitäts-Matrix** §Kritische Abhängigkeiten), wird er als erstes implementiert und mit Beispielen aus den 2 README-Examples (§1.2) sowie Edge-Cases (unsolvable, multi-solution) abgesichert.
3. **Validation auf 3 Layern:** Jede strikte Constraint wird auf Client + Server + DB redundant gesichert. Dies macht Tests gegen einzelne Layer wertvoll, statt Doppel-Tests.
4. **Mockup vor Implementierung:** README §1.1 verlangt Mockups als Doku-Bestandteil — sie werden separat erstellt (**Mockup-Briefings**) und sind die UI-Spezifikation, gegen die Blazor-Components gebaut werden.
5. **arc42 referenziert, dupliziert nicht:** Die Quell-Dokumente (**Use-Cases-Dokument**, **Funktionalitäts-Matrix**, **Validation-Regeln**, **ER-Modell**) sind die Single Source of Truth — arc42-Kapitel verlinken sie statt sie zu kopieren. So gibt es keine driftenden Mehrfach-Quellen.

Nächste Kapitel:

- [Kapitel 5 — Bausteinsicht](#): Detail der drei Layer mit Klassen, Schnittstellen, Verantwortungen.
- [Kapitel 6 — Laufzeitsicht](#): Vier Schlüssel-Szenarien (Puzzle-Save, Hint, Check, Pause/Resume) als Sequenz-Diagramme.

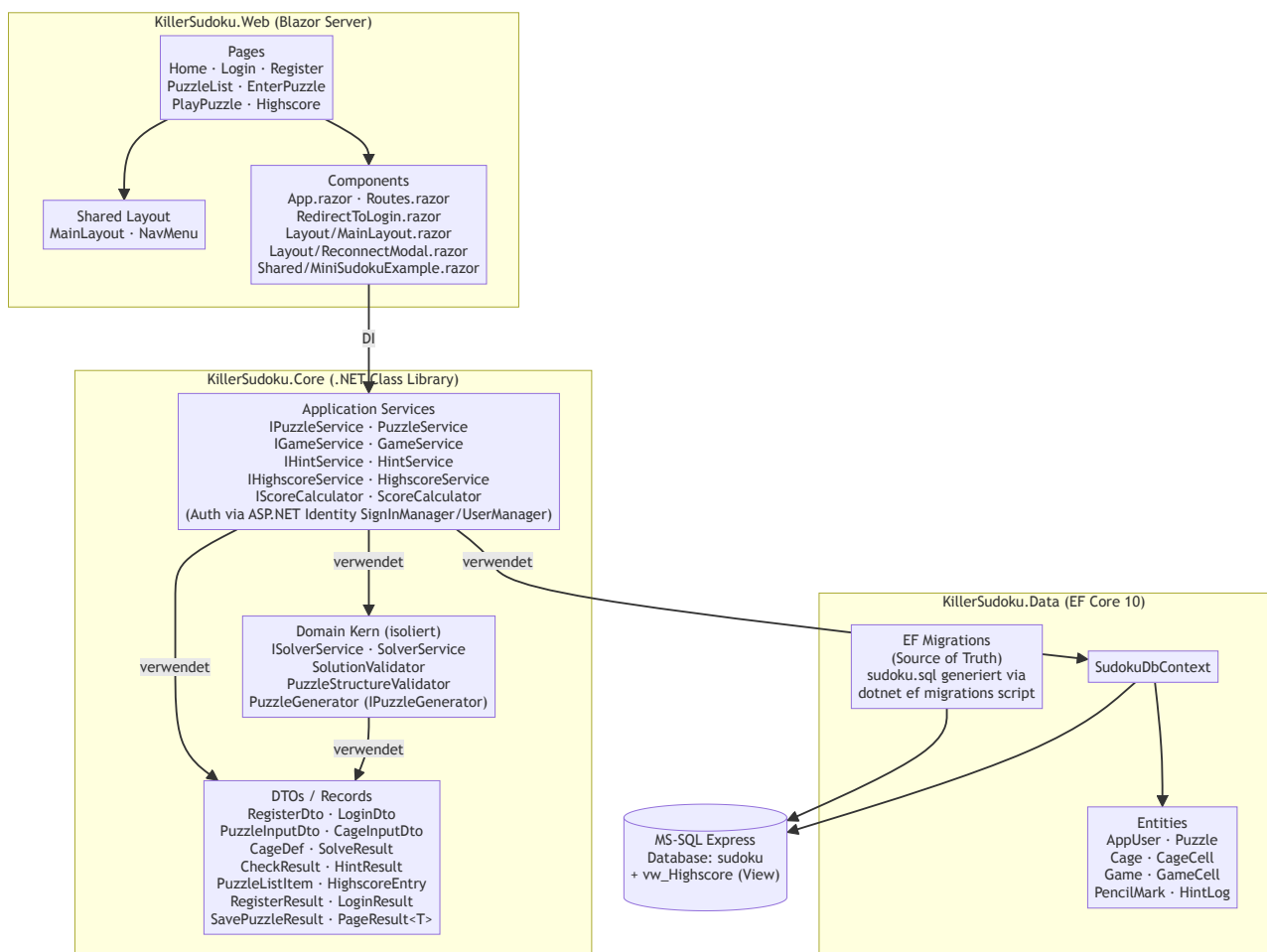
5 Bausteinsicht

arc42 v8.2 · Kapitel 5 · Killer Sudoku Bezug: [Kapitel 4](#) · **Funktionalitäts-Matrix** (autoritativ für Service-Signaturen) · **ER-Modell** · **Datenbank-Skript**

Dieses Kapitel zerlegt das System statisch in seine Bausteine. Die Service-Signaturen werden **nicht** dupliziert — sie stehen in **Funktionalitäts-Matrix** §Service-Interfaces. Hier wird die Architektur-Sicht beschrieben: Zweck, Schnittstellen, enthaltene Klassen, Abhängigkeiten.

5.1 Whitebox Gesamtsystem

Die Gesamtanwendung besteht aus drei Projekten/Assemblies (vorgesehene Aufteilung in der .NET-Solution):



Projekt-Verantwortlichkeiten kurz:

- `KillerSudoku.Web` — UI-Schicht; rein präsentational; keine Geschäftslogik; ruft Services per DI auf.
- `KillerSudoku.Core` — Application Services + isolierter Domain-Kern (Solver/Validator) + DTOs; **keine** EF-Core-Abhängigkeit im Domain-Kern.
- `KillerSudoku.Data` — Persistenz; EF-Core-DbContext, Entity-Mappings, Migrations. Wird von Application-Services verwendet, aber nicht von der UI direkt.

5.2 Baustein-Tabellen (Whitebox je Layer)

Für jeden Baustein: Zweck · Schnittstellen (eingehend/ausgehend) · enthaltene Klassen/Komponenten. Service-Signaturen siehe **Funktionalitäts-Matrix**.

5.2.1 KillerSudoku.Web — Presentation Layer

Aspekt	Details
Zweck	Blazor-Server-Anwendung; rendert UI, leitet User-Aktionen an Application-Services weiter, zeigt Ergebnisse.
Eingehende Schnittstelle	HTTP-Requests vom Browser (Routes aus Funktionalitäts-Matrix §Screen-Inventar S1–S8); SignalR-Verbindung für interaktive Updates.
Ausgehende Schnittstelle	Application-Services via DI: <code>IPuzzleService</code> , <code>IGameService</code> , <code>IHintService</code> , <code>IHighscoreService</code> . Auth direkt über ASP.NET-Identity- <code>SignInManager<AppUser></code> und <code>userManager<AppUser></code> (kein eigener Wrapper-Service).
Pages (@page)	<code>Home.razor</code> (S1, /) · <code>Auth/Register.razor</code> (S2, /register) · <code>Auth/Login.razor</code> (S3, /login) · <code>Puzzles.razor</code> (S4, /puzzles) · <code>EnterPuzzle.razor</code> (S5, /puzzles/new) · <code>PlayPuzzle.razor</code> (S6, /puzzles/{id}/play) · <code>Highscore.razor</code> (S7, /highscore).
Shared Components	<code>App.razor</code> · <code>Routes.razor</code> · <code>RedirectToLogin.razor</code> · <code>MainLayout.razor</code> · <code>ReconnectModal.razor</code> · <code>MiniSudokuExample.razor</code> · <code>Error.razor</code> · <code>NotFound.razor</code> .
Querschnitt	<code>[Authorize]</code> auf allen Pages außer S1/S2/S3 (V16); <code>EditForm</code> mit Antiforgery (V15); Razor- <code>@</code> -Encoding (V14).
Test-Strategie	bUnit-Component-Tests (Rendering, Event-Handler), E2E ohne Framework gemäß README §3.2.

Inline-UI: Die im class-diagram.md skizzierten Sub-Komponenten (`PuzzleGrid`, `Toolbar`, `CageEditor`, `HintButton`, `PauseButton`, `CheckSolutionButton`, `PencilMarkLayer`, `FilterBar`, `PuzzleCard`, `HighscoreTable`, `RegisterForm`, `LoginForm`, `RulesPanel`, `NavMenu`) sind aktuell **nicht als eigene .razor-Dateien implementiert** — ihre Markup-/Event-Handler-Logik lebt inline in den Pages (`PlayPuzzle.razor`, `EnterPuzzle.razor`, `Puzzles.razor`,

`Highscore.razor`, `Home.razor`, `Auth/Login.razor`, `Auth/Register.razor`). Dies hält den Component-Tree flach. Eine spätere Extraktion ist möglich, ohne API-Brüche.

5.2.2 KillerSudoku.Core — Application Services

Aspekt	Details
Zweck	Use-Case-Orchestrierung; konvertiert UI-Eingaben in DB-Operationen + Domain-Aufrufe; setzt Transaktions- und Autorisierungs-Grenzen.
Eingehende Schnittstelle	Service-Interfaces <code>IPuzzleService</code> , <code>IGameService</code> , <code>IHintService</code> , <code>IHighscoreService</code> — komplette Signaturen siehe Funktionalitäts-Matrix. Auth-Flows (UC02/UC03) verwenden direkt ASP.NET-Identity (<code>SignInManager<AppUser></code> , <code>userManager<AppUser></code>) ohne eigene Wrapper-Service-Schicht.
Ausgehende Schnittstelle	<code>ISolverService</code> (Domain) + <code>SudokuDbContext</code> (Data).
Enthaltene Services	<code>PuzzleService</code> (UC04/UC05/UC12) · <code>GameService</code> (UC06/UC09/UC10/UC13/UC14) · <code>HintService</code> (UC07) · <code>HighscoreService</code> (UC08) · <code>ScoreCalculator</code> (Singleton, pure Funktion — wird von <code>GameService.CompleteGameAsync</code> aufgerufen). UC02/UC03 (Register/Login) werden direkt über ASP.NET-Identity-Services (<code>SignInManager</code> , <code>userManager</code>) in den Razor-Pages umgesetzt.
UC-Mapping	Vollständig in Funktionalitäts-Matrix §Matrix-Tabelle.
Test-Strategie	Unit-Tests pro Service mit gemocktem <code>ISolverService</code> + <code>SudokuDbContext</code> (EF InMemory); Integration-Tests gegen echte MS-SQL-Test-DB.

HighscoreService — View-Status: Der Service liest aktuell mit LINQ-Joins über `Games` ⋈ `AppUser` ⋈ `Puzzle`. Die View `vw_Highscore` existiert zwar im SQL-Schema (`db/sudoku.sql`), wird aber vom `SudokuDbContext` **nicht** als Keyless-Entity gemappt. Switch ist offen — Schema-Reserve.

5.2.3 KillerSudoku.Core — Domain-Kern (Solver/Validator)

Aspekt	Details
Zweck	Reine Algorithmik für die vier Killer-Sudoku-Regeln aus README §1: (1) "Each row, column, and nonet contains each number exactly once", (2) "The sum of all numbers in a cage must match the small number printed in its corner", (3) "No number appears more than once in a cage", (4) "The solution must be unique".
Eingehende Schnittstelle	<code>ISolverService</code> — Methoden <code>Solve(givenValues, cages)</code> und <code>CountSolutions(givenValues, cages)</code> (Funktionalitäts-Matrix §Service-Interfaces).
Ausgehende Schnittstelle	Keine — pure C# ohne Framework- oder DB-Abhängigkeit.
Enthaltene Klassen	<code>SolverService</code> (Backtracking + MRV + Bit-Masken + Cage-Pruning, sealed) · <code>SolutionValidator</code> (Vollständigkeit → $\Sigma=405$ → Row/Col/Nonet → Cage-Sum/Duplikat für UC09) · <code>PuzzleStructureValidator</code> (Pre-Save-Check: Difficulty, Cage-Sum-Range, Coverage 81 Zellen für UC04) · <code>PuzzleGenerator</code> (Random-Generator für UC04 Editor) · <code>ScoreCalculator</code> (Score-Formel UC08/UC10). Die Hint-Strategien (Naked Single → Cage-Forced → Solver-Fallback) sind in <code>HintService</code> (Data-Layer) inline implementiert , nicht als eigenständige Domain-Klasse.
Performance-Anforderung	AC11.2: terminiert auch bei schweren Puzzles in < 2 s; bricht bei Zähl-Modus nach der 2. gefundenen Lösung ab.
Test-Strategie	Reine Unit-Tests mit den 2 README-Beispielen (§1.2) + Edge-Cases: unsolvable, multi-solution, vollständig vor-gelöst, minimal-clue. Höchste Test-Coverage-Priorität (siehe Funktionalitäts-Matrix §Kritische Abhängigkeiten).

5.2.4 KillerSudoku.Data — Persistence Layer

Aspekt	Details
Zweck	EF-Core-Persistenz; Mapping C#-Entities ↔ T-SQL-Tabellen aus Datenbank-Skript ; Query-Optimierung; View-Zugriff.
Eingehende Schnittstelle	<code>SudokuDbContext</code> (<code>DbSet<T></code> für jede Entity).
Ausgehende Schnittstelle	T-SQL gegen MS-SQL Express via ADO.NET-Provider.
Enthaltene Entities	<code>AppUser</code> · <code>Puzzle</code> · <code>Cage</code> · <code>CageCell</code> · <code>Game</code> · <code>GameCell</code> · <code>PencilMark</code> · <code>HintLog</code> . Schema-Quelle: Datenbank-Skript , ERD: ER-Modell .
View-Read-Model	<code>vw_Highscore</code> existiert im SQL-Schema (<code>db/sudoku.sql</code> Z. 215–231), wird vom aktuellen <code>SudokuDbContext</code> aber nicht als Keyless-Entity gemappt. <code>HighscoreService</code> liest stattdessen via LINQ-Join über <code>_db.Games</code> (siehe §5.2.2).
DB-Constraints (Defense-Layer)	Siehe §5.4 unten.
Test-Strategie	Integration-Tests mit Testcontainers (MS-SQL-Image) oder lokaler Test-DB; Constraint-Tests (INSERT mit Difficulty=4 muss fehlschlagen) gemäß ER-Modell §Ableitungen für Tests.

5.3 ER-Modell (Verweis + Einbettung)

Das vollständige ER-Modell mit Identity-Spalten und Nullable-Markierungen ist autoritativ in **ER-Modell**. Für die PDF-Lesbarkeit wird hier nur eine konzeptionelle Übersicht referenziert — die fachlichen Beziehungen lauten:

- `AppUser` (Identity-Basisklasse) 1:N `Puzzle` (CreatedBy)
- `AppUser` 1:N `Game`
- `Puzzle` 1:N `Cage` (1..*)
- `Cage` 1:N `CageCell` (jede Zelle 0–8 × 0–8 pro Puzzle eindeutig — durch Trigger erzwungen)
- `Game` 1:N `GameCell` , `PencilMark` , `HintLog`

Schlüssel-Begründungen (Auszug aus ER-Modell):

- `AppUser` statt `User` — `USER` ist reserviertes T-SQL-Keyword.
- `RowIdx` / `ColIdx` statt `Row` / `Col` — beides reserviert.
- **Keine Solution-Spalte** in `Puzzle` — UC04 wörtlich: *"No solution is recorded. Solutions must be calculated with an algorithm"*.
- `vw_Highscore` -VIEW existiert im SQL-Schema als Reserve; `HighscoreService` liest aktuell via LINQ-Join (single source of truth: `Game` -Tabelle).

- Pause-Mechanik via `IsPaused` + `PausedAt` + `TotalPausedSeconds` → erlaubt sauberen Resume mit `TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds`.

5.4 SQL-Schema-Übersicht

Vollständiges Schema in **Datenbank-Skript**. Folgende Objekte werden angelegt:

Tabellen (in Reverse-FK-Reihenfolge erzeugt):

- `AppUser` — Benutzer mit gehashtem Passwort
- ASP.NET Identity-Basis: nutzt `UserName`, `NormalizedUserName`, `Email`, `NormalizedEmail`, `PasswordHash`, `SecurityStamp`, `ConcurrencyStamp`, `Lockout-Felder`, `AccessFailedCount`.
- Filtered UNIQUE: `IX_AppUser_UserName` und `IX_AppUser_Email` (jeweils WHERE NOT NULL).
- **Keine** CHECK-Constraints für `UserName`-Non-Empty oder `Email`-Format im Schema — Validation läuft Client + Server (siehe **Validation-Regeln** V01/V02).
- Zusätzliche Spalte `CreatedAt` (`datetime2(0)`).
- `Puzzle` — Killer-Sudoku-Definition, **keine** Solution-Spalte
- FK: `CreatedById` → `AppUser(Id)`
- CHECK: `Difficulty BETWEEN 1 AND 3`
- `Cage` — Cage-Gruppe mit Soll-Summe
- FK: `PuzzleId` → `Puzzle(Id)` ON DELETE CASCADE
- CHECK: `Sum BETWEEN 1 AND 45`
- Index: `IX_Cage_PuzzleId`
- `CageCell` — Zelle in einem Cage (Composite PK)
- PK: `(CageId, RowIdx, ColIdx)`
- FK: `CageId` → `Cage(Id)` ON DELETE CASCADE
- CHECK: `RowIdx, ColIdx BETWEEN 0 AND 8`
- `Game` — Spiel-Session
- FK: `UserId` → `AppUser(Id)`, `PuzzleId` → `Puzzle(Id)`
- CHECK: `TimeSeconds` NULL oder ≥ 0 , `Score` NULL oder ≥ 0 , `HintsUsed` ≥ 0 , `TotalPausedSeconds` ≥ 0
- Indices: `IX_Game_UserId`, `IX_Game_PuzzleId`
- `GameCell` — Aktueller Spielzustand pro Zelle
- PK: `(GameId, RowIdx, ColIdx)`
- FK: `GameId` → `Game(Id)` ON DELETE CASCADE
- CHECK: `CellValue IS NULL OR BETWEEN 1 AND 9`
- `PencilMark` — Kandidaten-Annotationen (UC14)
- PK: `(GameId, RowIdx, ColIdx, MarkValue)`
- FK: `GameId` → `Game(Id)` ON DELETE CASCADE
- CHECK: `MarkValue BETWEEN 1 AND 9`
- `HintLog` — Audit-Log für Hint-Nutzung (UC07)

- FK: `GameId → Game(Id)` ON DELETE CASCADE
- Index: `IX_HintLog_GameId`

Index/Constraint-Spezialitäten:

- **Trigger** `trg_CageCell_UniquePerPuzzle` — verhindert, dass eine Zelle in mehreren Cages desselben Puzzles liegt. Wirft `THROW 50001` bei Verletzung (siehe **Datenbank-Skript** Zeilen 100–121).
- **Filtered Unique Index** `UX_Game_ActiveOnly` — `(UserId, PuzzleId)` WHERE `IsCompleted = 0`; setzt AC13.3 durch ("max 1 aktives Game pro User-Puzzle-Kombi").

View:

- `vw_Highscore` — JOIN über `Game ⋈ AppUser ⋈ Puzzle` für `IsCompleted = 1 AND Score IS NOT NULL`. Liefert `GameId · UserId · Username · PuzzleId · Difficulty · TimeSeconds · HintsUsed · Score · CompletedAt`. Wird von `IHighscoreService.GetTopAsync(limit)` für UC08 verwendet.

Nächstes Kapitel: [Kapitel 6 — Laufzeitsicht](#) — Sequenz-Diagramme der vier wichtigsten Szenarien (Puzzle-Save, Hint, Check-Solution, Pause/Resume).

6 Laufzeitsicht

arc42 v8.2 · Kapitel 6 · Killer Sudoku Bezug: [Kapitel 5](#) · **Use-Cases-Dokument** · **Funktionalitäts-Matrix** · **Validation-Regeln** · **ER-Modell**

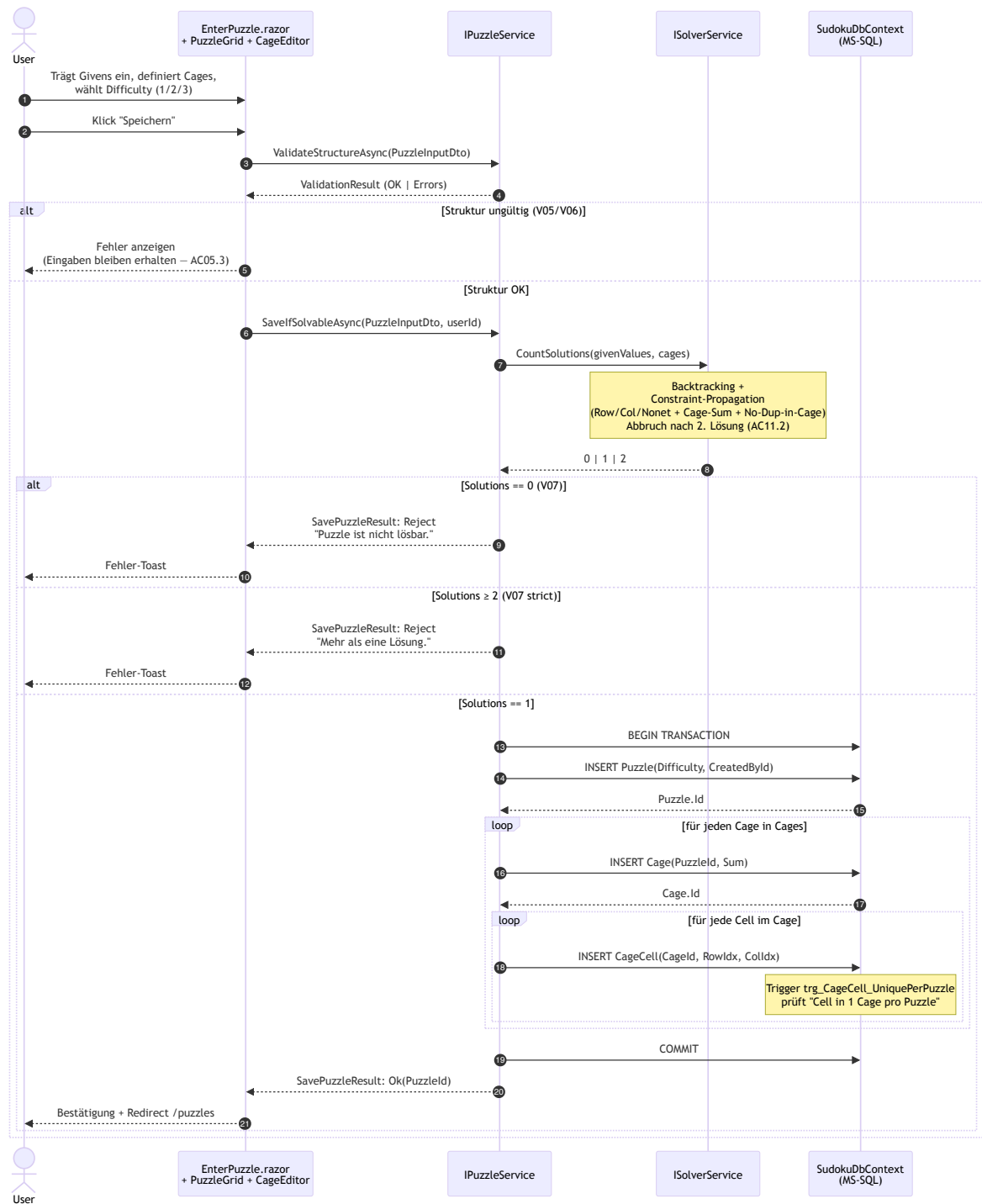
Dieses Kapitel beschreibt die wichtigsten **Laufzeit-Szenarien** als Sequenz-Diagramme. Service-Methoden-Namen entsprechen exakt dem Service-Interface-Block in **Funktionalitäts-Matrix** §Service-Interfaces. DB-Tabellen sind in **Datenbank-Skript** belegt.

Auswahl der Szenarien: Die vier dargestellten Use-Case-Flows decken die kritischen Architektur-Pfade ab — Save-with-Solver, Hint-Generation, Solution-Check (inkl. Sum-Fast-Fail) und Pause/Resume-State-Persistierung. Diese Pfade involvieren alle drei Layer + den Domain-Kern.

6.1 Szenario A — Puzzle anlegen und speichern (UC04 + UC05)

Ziel: User erstellt ein neues Killer-Sudoku, Server validiert Struktur, ruft Solver für Eindeutigkeits-Check, persistiert nur bei genau einer Lösung. Wörtliche README-Grundlage: *"The puzzle can only be saved if it is solvable"* (UC05) und *"The solution must be unique"* (§1).

Beteiligt: UC04 + UC05 · **V05** · **V06** · **V07** · Screen S5.



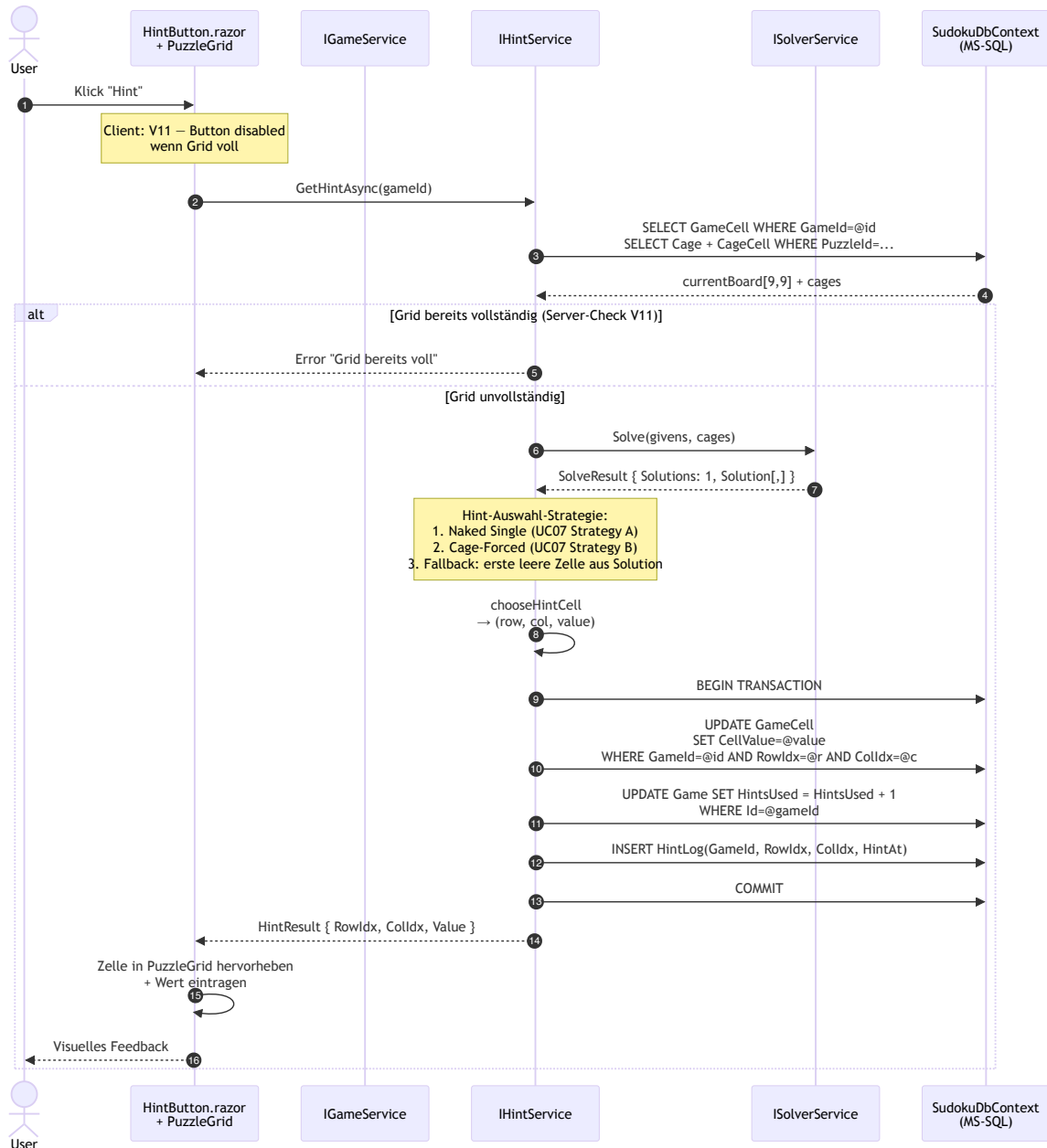
Wichtige Architektur-Eigenschaften:

- **Transaktionsklammer** liegt in `IPuzzleService.SaveIfSolvableAsync`, nicht in der UI.
- **Solver wird VOR jedem INSERT** ausgeführt — keine "Roll-Back-on-Failure"-Konstruktion nötig.
- **Keine Solution-Speicherung** — UC04 wörtlich: *"No solution is recorded"*; die Cage-Tabelle reicht für späteres Re-Solving.
- **DB-Trigger** ist Last-Line-of-Defense — falls die Application-Validierung Bugs hat, schützt der Trigger Datenintegrität (Defense-in-Depth).

6.2 Szenario B — Hint anfordern (UC07)

Ziel: User klickt "Hint", System bestimmt aus aktuellem Spielzustand eine Zelle + den korrekten Wert, persistiert Hint-Audit-Eintrag, inkrementiert `HintsUsed` (Score-relevant via UC08).

Beteiligt: UC07 · V11 · Screen S6 · `IHintService` + `ISolverService`.



Wichtige Architektur-Eigenschaften:

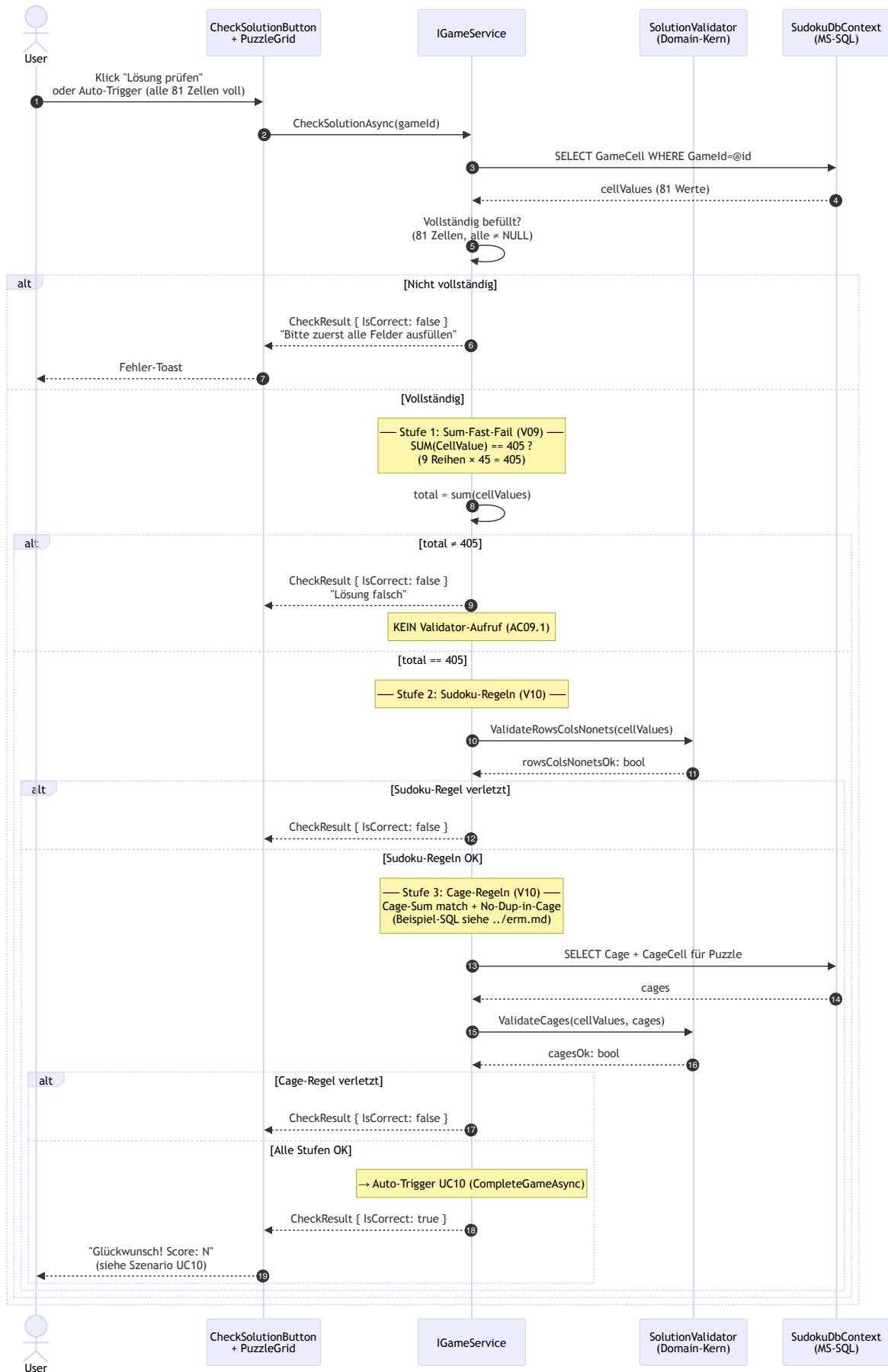
- **Solver wird stateless aufgerufen** — er bekommt den aktuellen Board-Zustand inkl. User-Eingaben; durch AC11.1 (Solver erkennt $0/1/\geq 2$ korrekt) ist garantiert, dass das gespeicherte Puzzle eine eindeutige Lösung hat, also `Solutions == 1` für vorherige Hint-Anfragen sicher ist (solange User-Eingaben nicht widersprüchlich sind — siehe UC07 AC07.3).
- **HintsUsed** wird atomar mit dem `GameCell`-Update und dem `HintLog`-INSERT in einer Transaktion erhöht — Audit + Score-Input bleiben konsistent.

- **Score-Wirkung:** Jeder Hint kostet 300 Punkte (Formel UC08 §High Score) — daher die Audit-Disziplin.
-

6.3 Szenario C — Lösung prüfen (UC09)

Ziel: User triggert Lösungsprüfung; System validiert in **drei Stufen**, sortiert nach Performance: (1) Sum-Check 405 als O(1)-Fast-Fail, (2) Row/Col/Nonet-Check, (3) Cage-Sum + Cage-No-Duplicate. README §2.3 wörtlich: *"Use the value for a 'simple' validation before checking the solution with an algorithm"*.

Beteiligt: UC09 · **V09** · **V10** · Screen S6 · `IGameService` .



Performance-Begründung (in Reihenfolge der drei Stufen):

1. **Sum-Check 405** — $O(81)$ Addition, kein Allokation. Filtert grobe Fehler (falsche Zahl, vergessene Zelle) sofort.
2. **Row/Col/Nonet** — 27×9 -Element-Distinct-Check, $O(243)$ ohne Set-Allokation via Bitmask.
3. **Cage-Sum + No-Duplicate** — Pro Cage einmal `cellValues` durchgehen, Set-basierter Duplikat-Check. Komplexität proportional zur Anzahl Cages $\times$ Cage-Größe ≤ 81 Operationen.

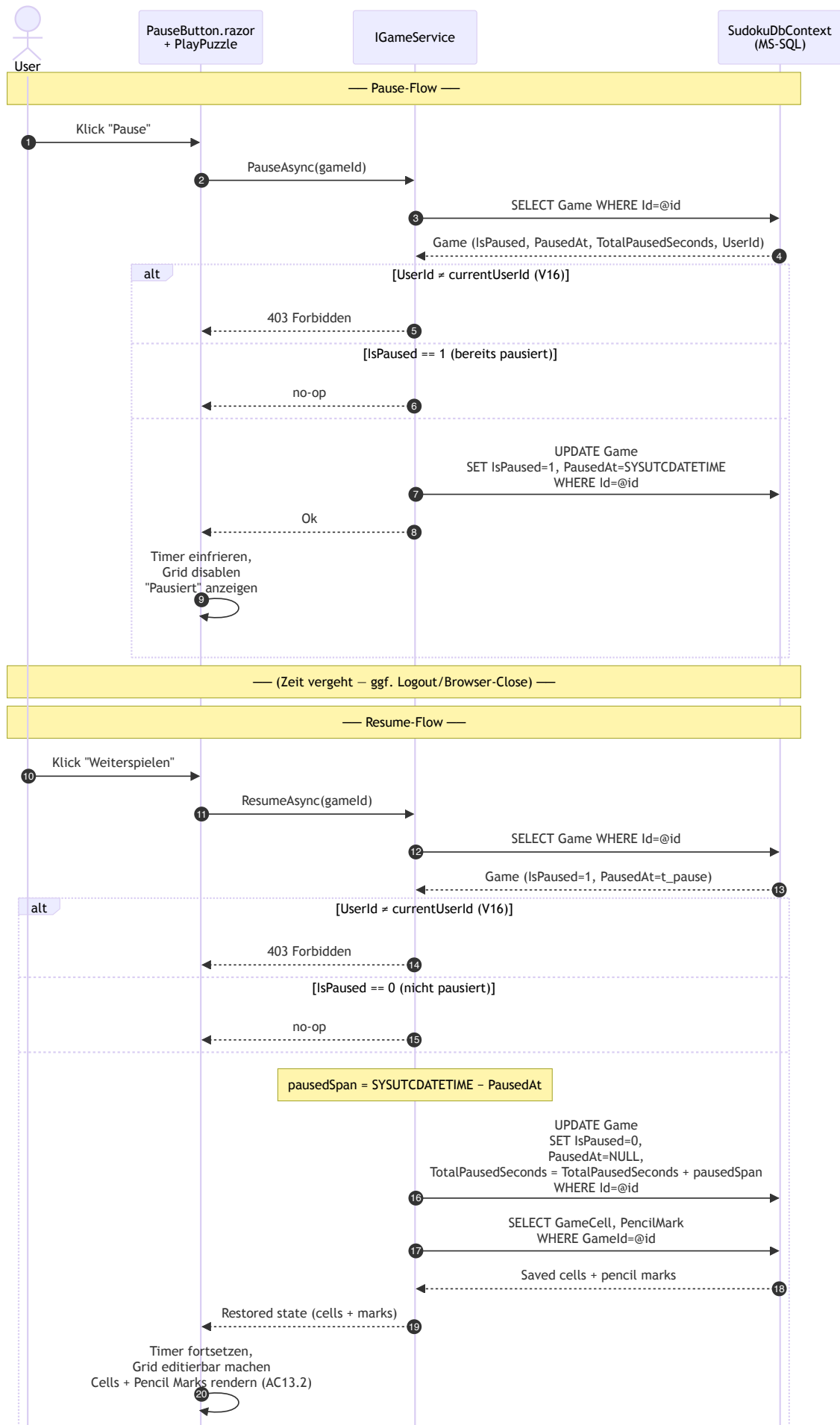
Die Sum-Check-Reihenfolge ist nicht Optimierung um der Optimierung willen — sie ist **wörtliche Anforderung** aus README §2.3 und Pattern für **alle** "simple before algorithmic" Validierungen im System.

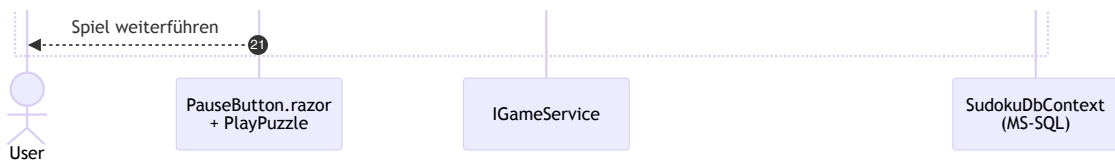
6.4 Szenario D — Pause und Resume (UC13)

Ziel: User pausiert ein laufendes Spiel; Server friert Timer ein und persistiert den Zustand; bei Resume wird der Timer um die Pausen-Dauer korrigiert weitergeführt. Formel aus **ER-Modell**:

`TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds` .

Beteiligt: UC13 · **V12** · Screen S6 · `IGameService.PauseAsync` / `ResumeAsync` .





Wichtige Architektur-Eigenschaften:

- **TotalPausedSeconds** wird inkrementell aufaddiert — mehrere Pause/Resume-Zyklen sind dadurch sauber unterstützt.
- **Timer-Berechnung** bleibt in der DB-Formel verankert (siehe Comment-Block in **Datenbank-Skript** Zeilen 125–127): bei Spiel-Abschluss (UC10) gilt `TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds`.
- **Filtered Unique Index** `UX_Game_ActiveOnly` (**ER-Modell**) garantiert AC13.3: ein User kann pro Puzzle nur **ein** nicht abgeschlossenes Game haben — verhindert Pause-im-Spiel-1 + Neu-Start-Spiel-2-vor-Resume-Doubles.
- **Auto-Save bei jedem Move** (UC13 Alt-Flow "Browser-Tab schliessen ohne Pause"): `IGameService.SetCellValueAsync` persistiert jeden Zell-Change sofort — daher ist auch ein impliziter "Pause durch Wegnavigation" verlustfrei.
- **Authorization (V16)**: jede Mutation prüft `Game.UserId == currentUserId` und antwortet mit 403 bei Mismatch.

Nächstes Kapitel: [Kapitel 7 — Verteilungssicht](#) (TBD) — Deployment-Diagramm: lokale Dev-Workstation, MS-SQL-Express-Instanz, Build-/Run-Konfiguration.

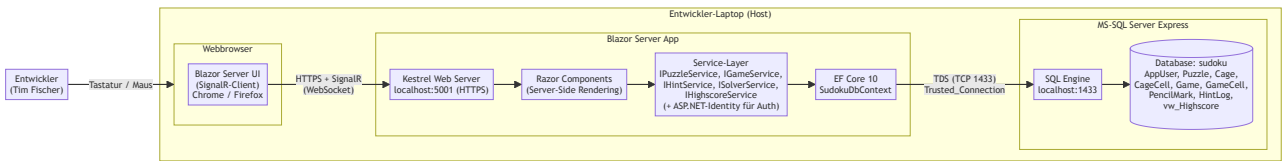
7 Verteilungssicht

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Dieses Kapitel beschreibt die physische Verteilung der Anwendung. Da die Aufgabenstellung explizit ein lokales Setup vorsieht (README §1.3: "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine**"), ist die Verteilungssicht bewusst einfach gehalten: Alle Komponenten laufen auf demselben Entwickler-Laptop.

§7.1 Deployment-Diagramm

Die folgende Verteilungssicht zeigt die Laufzeit-Knoten der Anwendung beim Wettbewerbs-Setup. Alle drei Komponenten (Browser, Blazor-Server-App, DB-Server) laufen auf demselben Host (Entwickler-Laptop).



Knoten-Inventar

Knoten	Typ	Adresse / Port	Technologie	Quelle
Entwickler-Laptop	Hardware-Host	localhost	macOS / Windows	README §1.3 ("on your machine")
Webbrowser	Client-Runtime	—	Chrome oder Firefox (modern HTML5)	Mockup-Briefs (Web-UI)
Blazor Server App	Anwendungs-Server	localhost:5001 (HTTPS)	.NET 10, Kestrel, Blazor Server (SignalR)	ER-Modell, Funktionalitäts-Matrix
MS-SQL Server Express	DB-Server	localhost:1433	MS-SQL Express, Datenbank sudoku	README §1.3 ("MS-SQL server"), Datenbank-Skript

Kommunikationspfade

Von	Zu	Protokoll	Bemerkung
Browser	Blazor Server App	HTTPS + WebSocket (SignalR)	Blazor Server hält permanente SignalR-Verbindung pro Tab/Circuit (siehe Funktionalitäts-Matrix)
Blazor Server App	MS-SQL Express	TDS (TCP 1433)	EF Core via <code>SqlConnection</code> , <code>Trusted_Connection</code> (Windows-Auth) bzw. lokaler Service-Account

Bemerkungen

- Es gibt **keine** Cloud-Komponenten, kein CDN, keine externen APIs — die Aufgabenstellung sieht ein rein lokales Setup vor.
- Die SignalR-Verbindung ist Folge des Blazor-Server-Modells (siehe [ADR-001](#)).
- Single-Host-Setup bedeutet: kein Reverse-Proxy, keine Lastverteilung, keine separate Auth-Komponente.

§7.2 Infrastruktur-Anforderungen (lokal)

Software-Voraussetzungen

Komponente	Version	Zweck	Quelle
.NET SDK	10.x	Build & Run der Blazor-Server-App	Stack-Definition ER-Modell, Funktionalitäts-Matrix
MS-SQL Server Express	aktuelle Version	Datenbank-Engine für <code>sudoku</code> -DB	README §1.3 ("MS-SQL server")
SQL Server Management Studio (SSMS)	beliebig	Ausführen von Datenbank-Skript (Schema-Setup), Tabellen-Inspektion	Operative Empfehlung
Visual Studio 2026 oder Visual Studio Code	beliebig	Entwicklungs-IDE	Empfehlung Entwickler-Werkzeug
Webbrowser	Chrome oder Firefox (modern)	Frontend-Runtime	Standard Blazor-Server-Anforderung

Hardware-Voraussetzungen

Da Blazor Server serverseitig rendert und der Solver eine reine In-Memory-Operation ist, sind die Hardware-Anforderungen moderat:

- Standard-Entwickler-Laptop (8 GB RAM, x64-CPU) ausreichend
- Disk: < 1 GB für SDK + Express + Source-Code + DB-Files

Konfiguration / Ports

Port	Dienst	Bemerkung
5001	Kestrel HTTPS (Blazor Server)	Default-Port .NET-Web-Templates
1433	MS-SQL Server Express	Default-Port SQL-Server

Datenbank-Setup

Die Datenbank-Erstellung erfolgt einmalig via SSMS oder `sqlcmd` :

```
# Beispiel mit sqlcmd (Windows / macOS-Crossplattform)
sqlcmd -S "(local)\SQLEXPRESS" -E -i db/sudoku.sql
```

Das Skript **Datenbank-Skript** ist **idempotent**: Es prüft auf `DB_ID('sudoku')` und droppt bestehende Objekte in Reverse-FK-Reihenfolge, bevor neu angelegt wird. Damit ist ein wiederholtes Ausführen ohne Konflikte möglich.

§7.3 Build & Run

Build

```
# Restore + Build
dotnet restore
dotnet build -c Release
```

Run (Entwicklung)

```
dotnet run --project source/src/KillerSudoku.Web
```

Die App ist anschliessend unter `https://localhost:5001` erreichbar.

Connection-String

```
Server=(local)\SQLEXPRESS;Database=sudoku;Trusted_Connection=True;TrustServerCertificate=
```

Der Connection-String wird via `appsettings.json` / `appsettings.Development.json` oder Umgebungsvariable konfiguriert. Trusted Connection (Windows-Auth) ist auf dem Wettbewerbs-Laptop ausreichend; Cross-Plattform-Setups (macOS) verwenden Username/Password über `User Id=... ;Password=...`.

Sicherheit: Auch im lokalen Setup wird der Connection-String nicht in den Source-Code hardkodiert (siehe [Kapitel 8 §8.4 Logging](#) und **V03**). `TrustServerCertificate=True` ist akzeptabel, da kein Netzwerk-Hop über das lokale Setup hinaus stattfindet.

Tests ausführen

```
dotnet test
```

Test-Frameworks gemäss [ADR-011](#):

- Unit-Tests (xUnit) — Solver-Logik, Service-Methoden, Validation
- Component-Tests (bUnit) — Razor-Components (**Funktionalitäts-Matrix**)
- Integration-Tests (xUnit + `WebApplicationFactory`) — DB + Auth + Service-End-to-End
- E2E-Tests (Playwright .NET) — Kritische User-Flows (UC01, UC03, UC06, UC09)

§7.4 Submission-Artefakte

ZIP-Inhalt (gemäss README §1.4)

README §1.4 fordert wörtlich:

"All deliverables must be submitted in a zip file named `AppDev_Name_FirstName.zip`. The deliverables are the documentation including the test protocol, executable files, the source code and the database script."

Daraus leitet sich folgende Submission-Struktur ab:

```

AppDev_Fischer_Tim.zip
├── doc/
│   ├── arc42.pdf (Kapitel 01–12 als ein PDF)
│   ├── mockups/ (PNG-Exports aus Figma)
│   └── test-protocol.pdf
├── source/ (.NET 10 Solution)
│   ├── KillerSudoku.slnx
│   ├── src/
│   │   ├── KillerSudoku.Web/ (Blazor Server App)
│   │   ├── KillerSudoku.Core/ (Solver, Domain, Services, DTOs)
│   │   └── KillerSudoku.Data/ (EF Core, Entities, DbContext, Service-Impl)
│   ├── tests/
│   │   ├── KillerSudoku.UnitTests/ (xUnit)
│   │   ├── KillerSudoku.IntegrationTests/ (xUnit + Testcontainers-MSSQL)
│   │   ├── KillerSudoku.ComponentTests/ (bUnit)
│   │   └── KillerSudoku.E2ETests/ (Playwright .NET)
│   ├── bin/
│   │   └── Release/net10.0/ (Build-Output, ausführbare Files)
│   ├── db/
│   │   └── sudoku.sql (DB-Skript)

```

Build-Output (Executables)

Das README fordert "executable files". Für .NET 10 + Blazor Server bedeutet dies:

```
dotnet publish source/src/KillerSudoku.Web -c Release -o bin/Release/net10.0
```

Der Publish-Output enthält:

- `KillerSudoku.Web.dll` (Haupt-Assembly)
- `KillerSudoku.Web.exe` (Windows-Launcher, falls auf Windows publiziert)
- Alle Dependency-DLLs
- `appsettings.json` (mit Connection-String-Slot)
- `wwwroot/` (statische Assets, CSS, JS)

Starten der publizierten App:

```
cd bin/Release/net10.0
dotnet KillerSudoku.Web.dll
```

Datenbank-Skript

`db/sudoku.sql` (siehe **Datenbank-Skript**) ist im ZIP enthalten und entspricht README §1.3:

"Create for each table a SQL statement and save them in a script called **Datenbank-Skript**. Don't forget to create the primary and foreign key constraints."

Das Skript erstellt:

- Datenbank `sudoku` (idempotent)
- Tabellen `AppUser`, `Puzzle`, `Cage`, `CageCell`, `Game`, `GameCell`, `PencilMark`, `HintLog`
- PK/FK/CHECK/UNIQUE-Constraints
- Trigger `trg_CageCell_UniquePerPuzzle` (siehe [ADR-004](#))
- View `vw_Highscore` (siehe [ADR-003](#))

Dokumentation

README §2.6 fordert wörtlich folgende Dokumentations-Sektionen:

README-Anforderung	Abgedeckt in
"Mockup"	Mockup-Briefings + Mockup-Verzeichnis
"Database diagram"	ER-Modell (Mermaid ERD) + Kapitel 5 Bausteinsicht
"Class diagram"	Kapitel 5 Bausteinsicht — Service-Interfaces aus Funktionalitäts-Matrix
"Additionally chosen use cases"	Use-Cases-Dokument UC12–UC14 + ADR-014
"Validation rules"	Validation-Regeln (V01–V16)
"Test protocol"	Test-Protokoll + Test-Protokoll

README §2.6 weiter: "For the delivery, create a PDF document." Die arc42-Kapitel werden zu einem PDF zusammengeführt (Submission-Format).

Termin-Constraints

README §1.4: "Note that the submission of the planned test cases must take place **by 12 o'clock**." Der Test-Plan ist in **Test-Protokoll** dokumentiert und wird vor 12:00:00 abgegeben (siehe Qualitätsziel Q3 in [Kapitel 1 §1.2](#)).

§7.5 Container-Deployment (Single-Image Docker)

Zusätzlich zur nativen Variante (§7.1–§7.3) wird ein **containerisiertes Single-Image-Deployment** ausgeliefert. Es enthält Webserver, App und Datenbank in einem Image — der Prüfer kann mit

einem einzigen `docker run` die gesamte Anwendung starten, ohne MS-SQL Server lokal installieren zu müssen.

Siehe [ADR-015](#) für die Begründung der Single-Image-Wahl (statt klassischem Multi-Service-Compose).

Deployment-Diagramm (Container)

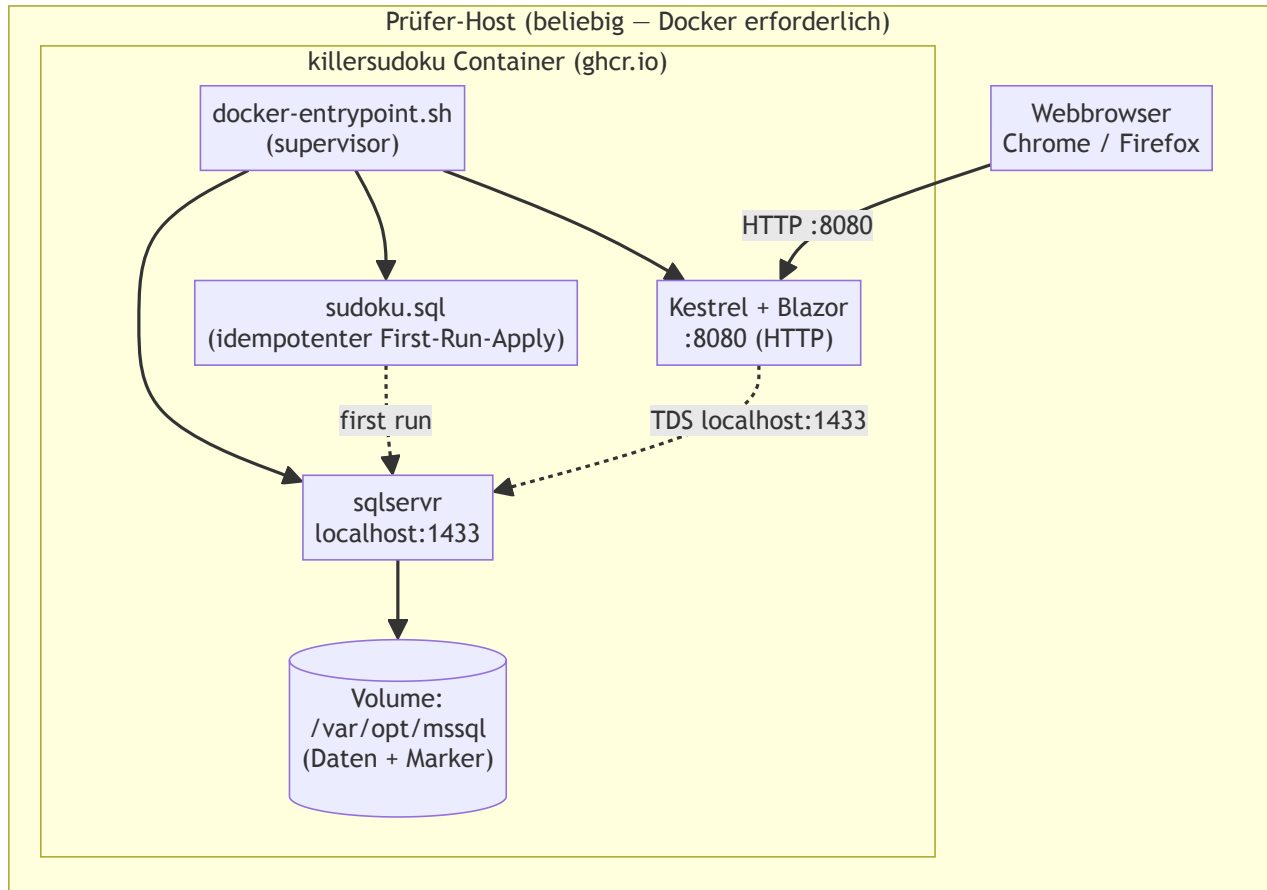


Image-Inhalt

Layer	Quelle	Zweck
Base	<code>mcr.microsoft.com/mssql/server:2022-latest</code> (Ubuntu 22.04)	DB-Engine + EULA-Akzeptanz
<code>aspnetcore-runtime-10.0</code>	<code>packages.microsoft.com</code>	.NET 10 ASP.NET Runtime für Blazor Server
<code>mssql-tools18</code>	MS-Repo	<code>sqlcmd</code> für Schema-Apply beim ersten Start
<code>/app/</code>	Multi-Stage Build (Stage 1)	<code>dotnet publish</code> Output des Web-Projekts
<code>/docker-init/sudoku.sql</code>	Repo <code>db/sudoku.sql</code>	Schema-Skript (DDL + Trigger + View)
<code>/docker-entrypoint.sh</code>	Repo-Root	Startup-Orchestrierung

Startup-Sequenz

`docker-entrypoint.sh` führt beim Container-Start folgende Schritte aus (atomar, vollständige Konvergenz):

1. `sqlservr` als Background-Prozess starten
2. Bis zu 60× `sqlcmd -Q "SELECT 1" -Ping (≤ 2 Min)` — wartet auf DB-Readiness
3. Falls Marker-File `/var/opt/mssql/.killersudoku-schema-applied` **nicht** existiert: **Datenbank-Skript** ausführen und Marker setzen
4. `dotnet KillerSudoku.Web.dll` als zweiter Hintergrund-Prozess starten
5. `wait -n` blockiert bis SQL oder App terminiert → dann sauberer SIGTERM an beide Children

Build & Run

Option A — Pull vom Registry (schnellster Pfad für Prüfer)

```
docker pull ghcr.io/tim-fischer-zh/killer-sudoku:latest
docker run -d \
  --name killersudoku \
  -p 8080:8080 \
  -e MSSQL_SA_PASSWORD='Sudoku!Strong#Pass#2026' \
  -v killersudoku-data:/var/opt/mssql \
  ghcr.io/tim-fischer-zh/killer-sudoku:latest

# Browser → http://localhost:8080
```

Option B — Lokal aus Sources bauen

```
# Repo-Root als Build-Context (enthält source/ + db/ + docker-entrypoint.sh)
docker build -t killersudoku:local .
docker run -d -p 8080:8080 \
  -e MSSQL_SA_PASSWORD='Sudoku!Strong#Pass#2026' \
  killersudoku:local
```

Option C — `docker compose` (empfohlen)

```
docker compose up --build
```

Compose-Datei: **docker-compose.yml** — bindet Port 8080 (App) und 1433 (DB, optional für externe SSMS-Inspektion), persistiert SQL-Daten in einem Named Volume.

Ports

Port	Zweck	Default-Mapping
8080	Kestrel HTTP (Blazor Server)	<code>-p 8080:8080</code>
1433	MS-SQL (intern + optional extern für SSMS)	<code>-p 1433:1433</code> (kann weggelassen werden)

Hinweis: Im Container läuft Kestrel auf HTTP (`:8080`), nicht HTTPS. Ein TLS-Terminator (z.B. Traefik/Caddy) ist nicht im Image enthalten — für die Submission-Bewertung ist Localhost-HTTP ausreichend. Die Auth-Cookies bleiben `HttpOnly + SameSite=Lax` (siehe **V03**) — der `Secure` -Flag ist im Container-Lauf nur unter HTTPS-Reverse-Proxy zu setzen.

Environment-Variablen

Variable	Default	Zweck
<code>MSSQL_SA_PASSWORD</code>	<code>Sudoku!Strong#Pass#2026</code>	SA-Passwort (Strong-Password-Policy von MS-SQL muss erfüllt sein)
<code>MSSQL_PID</code>	<code>Express</code>	Edition (Free-Tier für Submission)
<code>ACCEPT_EULA</code>	<code>Y</code>	MS-SQL Lizenz-Akzeptanz (Pflicht)
<code>ASPNETCORE_URLS</code>	<code>http://+:8080</code>	Kestrel-Binding
<code>ConnectionStrings__Sudoku</code>	siehe Dockerfile	Override möglich für Tests

Persistenz

Die DB-Files (`mdf` / `ldf`) liegen in `/var/opt/mssql` im Container. Ein Named-Volume `killersudoku-data` (oder ein Bind-Mount) erhält den Spielstand zwischen Container-Neustarts. Das Marker-File `/var/opt/mssql/.killersudoku-schema-applied` verhindert mehrfaches Schema-Apply.

CI / Registry-Push

Die GitHub-Actions-Pipeline `.github/workflows/deploy.yml` baut bei jedem `push` auf `main` das Image und pusht es nach `ghcr.io/tim-fischer-zh/killer-sudoku` mit Tags:

- `:latest` (auf default-Branch)
- `:<git-short-sha>` (immutable per Commit)

Im Anschluss führt derselbe Workflow das Production-Deployment auf der VPS aus (siehe §7.6).

Submission-Inhalt (Update zu §7.4)

Zusätzlich zum ZIP-Inhalt aus §7.4 enthält das Repository (und damit `src/` im ZIP):

- `Dockerfile` (Repo-Root)
- `docker-entrypoint.sh` (Repo-Root)
- `docker-compose.yml` (Repo-Root, lokal)
- `docker-compose.prod.yml` (Repo-Root, Production — siehe §7.6)
- `.dockerignore` (Repo-Root)
- `.github/workflows/deploy.yml`

Der Prüfer kann damit zwischen den drei Optionen (Pull / Local-Build / Compose) wählen.

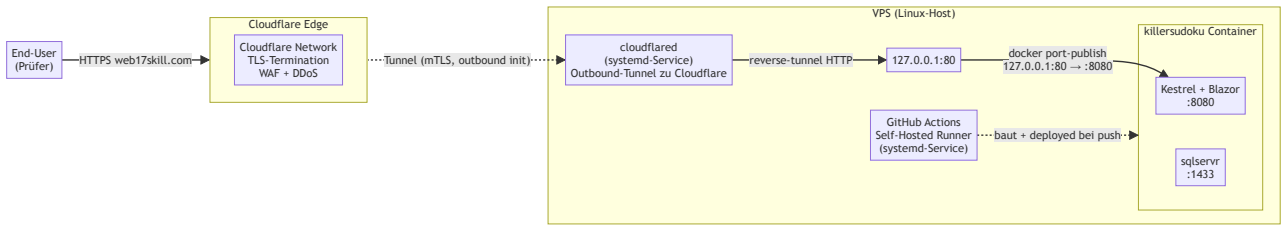
§7.6 Production-Deployment (Bonus: Live-Demo unter web17skill.com)

Über das von der Aufgabenstellung geforderte lokale Setup (§7.1–§7.5) hinaus wird die Anwendung **zusätzlich** auf einer öffentlich erreichbaren VPS-Instanz betrieben, damit der Prüfer einen Live-Stand der App ohne lokale Installation aufrufen kann. Die Produktions-Topologie ist **kein** Ersatz für das lokale Setup — sie ergänzt es.

Live-Endpoint

Item	Wert
Domain	web17skill.com
Protokoll	HTTPS (TLS-Termination bei Cloudflare)
TLS-Zertifikat	Cloudflare-managed (automatisch)
Health-Endpoint	https://web17skill.com/health

Topologie



Knoten-Inventar (Production)

Knoten	Typ	Adresse	Quelle
Cloudflare Edge	Managed CDN/Proxy	web17skill.com	externer Dienst
VPS	Linux-Host (Docker, systemd)	privat, nicht-öffentlich erreichbar	Tim Fischer Setup
cloudflared	systemd-Service	bindet 127.0.0.1:80 als Tunnel-Origin	Cloudflare-Dokumentation
GitHub Actions Self-Hosted Runner	systemd-Service	derselbe VPS-Host wie Cloudflared	GitHub-Dokumentation
killersudoku-Container	Docker-Container	127.0.0.1:80 → 8080	docker-compose.prod.yml

Kommunikationspfade (Production)

Von	Zu	Protokoll	Bemerkung
End-User-Browser	Cloudflare	HTTPS (TLS 1.3)	Cloudflare-WAF und DDoS-Mitigation aktiv
Cloudflare	<code>cloudflared</code> auf VPS	mTLS-Tunnel (outbound initiiert)	Kein eingehender Port auf der VPS offen — Tunnel terminiert HTTPS bei Cloudflare und forwarded plain HTTP an <code>127.0.0.1:80</code>
<code>cloudflared</code>	Container	HTTP <code>127.0.0.1:80</code>	Docker-Port-Publish bindet bewusst nur auf Loopback — Container ist nicht von außen direkt erreichbar
Container-intern	sqlservr	TDS <code>localhost:1433</code>	DB-Port nicht publiziert — interner Zugriff nur über Loopback im Container

Warum Cloudflare-Tunnel statt offener Port?

Aspekt	Tunnel (gewählt)	Klassischer Reverse-Proxy mit offenem Port
Eingehende Firewall-Regeln auf VPS	keine nötig	Port 80/443 müssen offen sein
TLS-Zertifikat	Cloudflare-managed (auto-renew)	selbst-verwaltet (Let's Encrypt o.ä.)
DDoS-Schutz	inkludiert	nicht inkludiert
IP-Adresse der VPS sichtbar	nein	ja

Der Tunnel ist **outbound initiiert** — die VPS hält die Verbindung zu Cloudflare offen, nicht umgekehrt. Das bedeutet, dass auf der VPS **keine eingehenden Ports** in der Firewall geöffnet werden müssen.

`docker-compose.prod.yml` — Production-Unterschiede

Die Production-Compose (**`docker-compose.prod.yml`**) unterscheidet sich an drei Stellen von der lokalen Compose (siehe §7.5):

Aspekt	Lokal (<code>docker-compose.yml</code>)	Production (<code>docker-compose.prod.yml</code>)
App-Port-Bind	<code>8080:8080</code> (alle Interfaces)	<code>127.0.0.1:80:8080</code> (nur Loopback, weil <code>cloudflared</code> lokal terminiert)
SQL-Port-Bind	<code>127.0.0.1:1433:1433</code> (lokale SSMS-Inspektion)	nicht publiziert (keine externe DB-Inspektion in Produktion)
Image-Quelle	lokaler <code>build:</code> Context	gezogen aus GHCR (<code>pull_policy</code> nicht gesetzt — Runner baut lokal und compose nutzt das Image direkt aus dem lokalen Daemon)

CI-Pipeline (`.github/workflows/deploy.yml`)

Der Workflow läuft auf `runs-on: self-hosted` — also direkt auf der VPS. Vier Schritte:

1. **Checkout** des Repos auf der VPS
2. `docker build` — lokal auf der VPS (Layer-Cache bleibt zwischen Runs erhalten)
3. **Push** der Image-Tags `:latest` und `:<short-sha>` nach GHCR (für Versionierung und ggf. spätere Rollbacks)
4. `docker compose -f docker-compose.prod.yml up -d --remove-orphans` — startet/ersetzt den Container; das Image liegt nach `docker build` bereits im lokalen Daemon, kein erneuter Pull nötig

Abschließend führt der Workflow einen **Smoke-Test** durch (`curl https://web17skill.com/health` mit Retry über 120 s) und schlägt fehl, falls der Live-Endpoint nicht innerhalb von 2 Minuten gesund antwortet.

Required Secrets im GitHub-Repository

Secret	Zweck
<code>GHCR_TOKEN</code>	Classic-PAT mit Scopes <code>write:packages</code> + <code>read:packages</code> für GHCR-Push
<code>MSSQL_SA_PASSWORD</code>	SA-Passwort des Production-SQL-Servers (Strong-Password-Policy)

Beide werden im Workflow als Job-Env-Variablen exponiert und vom Container über `.env` bzw. das DI-Setup konsumiert.

Trennung Lokal ↔ Production

Eigenschaft	Lokal (Spec §1.3)	Production (Bonus)
Pflicht laut Aufgabenstellung	ja	nein
Adresse	<code>localhost:5001</code> (native) bzw. <code>localhost:8080</code> (Compose)	<code>https://web17skill.com</code>
TLS	self-signed Dev-Cert	Cloudflare-managed (echtes Zertifikat)
Persistenz	Named Volume oder lokaler SQL-Server	Named Volume <code>mssql-data</code> auf VPS
Deploy-Mechanismus	manueller <code>dotnet run</code> / <code>docker compose up</code>	Push auf <code>main</code> → automatischer CI-Build + Deploy

Beide Setups verwenden **dasselbe Image** und **dasselbe Schema** — Production ist nur ein anderer Lauf-Modus, kein architektonischer Unterschied. Die Bewertung der Submission stützt sich ausschließlich auf das lokale Setup; die Live-Variante dient als Bonus für eine bequemere Demonstration.

Verweise

- [Kapitel 2 — Randbedingungen](#)
- [Kapitel 5 — Bausteinsicht](#)
- [Kapitel 8 — Querschnittliche Konzepte](#)
- [Kapitel 9 — Architekturentscheidungen](#)
- **Funktionalitäts-Matrix** — Service-Layer-Inventar
- **Datenbank-Skript** — DB-Schema
- **Aufgabenstellung** — Aufgabenstellung

8 Querschnittliche Konzepte

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Dieses Kapitel sammelt Konzepte, die mehrere Bausteine oder Schichten der Architektur betreffen. Jedes Konzept verweist zurück auf seine UC- und Validation-Anker; eigentliche Regel-Listen werden nicht dupliziert, sondern referenziert.

§8.1 Authentifizierung & Autorisierung

Konzept

Die Anwendung verwendet **ASP.NET Core Identity** als Authentifizierungs-Provider und Cookie-basierte Sessions. Login ist Pflicht für alle nicht-öffentlichen Use Cases (UC04–UC14). UC01 (Read Rules) ist die einzige öffentliche Seite.

Anforderungen aus README

- README §2.1 UC2: "To use the application, a login **is required**."
- README §2.1 UC3: "The user logs in."

Implementations-Konzept

Aspekt	Lösung
Identity-Provider	ASP.NET Core Identity (Standard <code>IdentityUser</code> -Schema, lokale Tabelle <code>AppUser</code>)
Passwort-Hashing	ASP.NET Identity Default (PBKDF2 mit HMAC-SHA256, 100 000 Iterationen) — siehe V03
Session-Mechanismus	Cookie (HttpOnly + Secure + SameSite=Lax — siehe §8.9)
Rate-Limiting Login	5 Fehlversuche / 5 Minuten — siehe V04
Geschützte Routen	<code>[Authorize]</code> -Attribut auf Razor-Pages bzw. Components — siehe V16
Cross-User-Mutation	Server prüft <code>game.UserId == currentUserId</code> vor jeder schreibenden Aktion — siehe V16

Validation-Anker

- V03 — Passwort
- V04 — Login Rate-Limit
- V16 — Authorization

Use-Case-Anker

- UC02 — Create User
- UC03 — Login

§8.2 Eingabe-Validierung (Defense in Depth)

Konzept

Alle User-Inputs werden auf **mindestens zwei Layern** geprüft:

1. **Client** (Blazor `EditForm` + Data Annotations + JS-Pattern)
2. **Server** (Service-Layer)
3. **Datenbank** (CHECK / UNIQUE / FK / Trigger — letzte Verteidigungslinie)

Diese Schichtung folgt der "Defense in Depth"-Konvention aus **Validation-Regeln**:

"Defense in Depth: Kritische Regeln werden auf mindestens 2 Layern geprüft. DB-Constraints sind die letzte Verteidigungslinie."

Übersicht

Die vollständige Regel-Liste mit allen 16 Regeln (V01–V16) ist in **Validation-Regeln** dokumentiert. Hier nur die Layer-Zuordnungs-Heuristik:

Regel-Kategorie	Client	Server	DB
Format (Username, Email, Pattern)	✓	✓	partiell (UNIQUE, LIKE)
Range (Difficulty, CellValue, MarkValue)	✓	✓	✓ (CHECK)
Solvability (UC05)	—	✓ (Solver)	—
Auth & Authorization	—	✓	partiell (FK)
XSS-Schutz	—	✓ (Razor-Auto-Encoding)	—
CSRF-Schutz	—	✓ (Antiforgery)	—

Validation-Anker

Vollständige Liste aller Regeln: **Validation-Regeln** V01–V16.

§8.3 Fehlerbehandlung

Konzept

Fehler werden auf zwei semantische Klassen aufgeteilt:

1. **Erwartete Fachfehler** (z.B. Username vergeben, Puzzle unlösbar, falsche Credentials) → strukturierte `Result<T>` - bzw. Either-Typen
2. **Unerwartete System-Fehler** (DB-Down, NullRef, etc.) → ASP.NET-Exception-Middleware → generische Fehlerseite

Strukturierte Result-Types

Service-Methoden geben statt Exceptions strukturierte Result-Typen zurück (siehe Service-Signaturen in **Funktionalitäts-Matrix**):

Result-Type	Verwendet von	Quelle
<code>IdentityResult</code>	<code>UserManager<AppUser>.CreateAsync</code>	UC02 (ASP.NET-Identity-Standard-Result)
<code>SignInResult</code>	<code>SignInManager<AppUser>.PasswordSignInAsync</code>	UC03 (ASP.NET-Identity-Standard-Result)
<code>ValidationResult</code>	<code>IPuzzleService.ValidateStructureAsync</code>	UC04
<code>SavePuzzleResult</code>	<code>IPuzzleService.SaveIfSolvableAsync</code>	UC05
<code>CheckResult</code>	<code>IGameService.CheckSolutionAsync</code>	UC09
<code>HintResult</code>	<code>IHintService.GetHintAsync</code>	UC07
<code>SolveResult</code>	<code>ISolverService.Solve</code>	UC11

Generische Login-Fehlermeldung

Bei falschen Credentials liefert UC03 absichtlich **keine** Auskunft darüber, ob der Username existiert (Security-Pattern gegen Account-Enumeration):

"Username oder Passwort falsch."

Siehe **V03** und UC03 AC03.1:

"AC03.1: Falsche Credentials → keine Auskunft darüber, ob Username existiert"

Use-Case-Anker

Alternative Pfade (`Alt`) sind pro UC in **Use-Cases-Dokument** dokumentiert.

§8.4 Logging

Konzept

Logging via `Microsoft.Extensions.Logging` (Standard-Abstraktion in .NET 10). Konkreter Provider: Console + File (Debug-Builds) / Console + EventLog (Release auf Windows).

Was geloggt wird

Ereignis	Log-Level	Begründung
Anwendungs-Start / Shutdown	Information	Operative Sichtbarkeit
Fehlgeschlagene Login-Versuche	Warning	Rate-Limit-Anker (siehe V04)
Solver-Aufrufe mit Dauer > 1s	Warning	Performance-Boundary aus UC11 AC11.2 (" < 2 Sekunden")
Puzzle-Save (Solvability-Check Ergebnis: 0/1/ ≥ 2)	Information	Audit für UC05
Unbehandelte Exceptions	Error	Standard
HTTP-Requests (Method, Path, Status, Duration)	Information	ASP.NET-Built-In Request-Logging

Was NICHT geloggt wird (kritisch)

- **Passwörter** — weder Klartext noch Hash, weder in Request-Bodies noch in Trace-Logs (siehe **V03**: "NIE im Klartext loggen/serialisieren")
- **Session-Cookies / Antiforgery-Token** — kein Logging von Request-Headers oberhalb von Debug-Level
- **PII über Username/Email hinaus** — UC erfordert keine weitergehenden Daten (siehe ERD in **ER-Modell**)

Validation-Anker

- V03

§8.5 Solver-Domänen-Konzept

Konzept

Der Solver (UC11) ist die zentrale Logik-Komponente und das **erste Qualitätsziel** der Architektur (siehe [Kapitel 1 §1.2 Q1](#)). Er wird daher als **Pure Function** im Domain-Layer implementiert:

- **Keine** Abhängigkeit auf `DbContext` oder IO
- **Keine** Abhängigkeit auf ASP.NET-Infrastruktur
- Eingabe: `int[,] givenValues` + `IReadOnlyList<CageDef> cages`
- Ausgabe: `SolveResult { Solutions, Solution? }`

Algorithmik

Aus **UC11**:

"Solver implementiert Backtracking mit Constraint-Propagation:

- Standard-Sudoku-Constraints (Row/Col/Nonet)
- Killer-Cage-Constraints (Summe + keine Duplikate in Cage)"

Der Solver bricht nach der **zweiten** gefundenen Lösung ab — siehe [ADR-010](#).

Aufrufer

Aufrufer	UC	Verwendung
<code>IPuzzleService.SaveIfSolvableAsync</code>	UC05	<code>CountSolutions == 1</code> ⇒ Puzzle wird gespeichert
<code>IHintService.GetHintAsync</code>	UC07	Solver liefert Solution; Hint-Strategie wählt Zelle (siehe ADR-013)
<code>IGameService.CheckSolutionAsync</code>	UC09	Optional: Solver bestätigt User-Eingabe vs. eindeutige Lösung

Testbarkeit

Da der Solver keine externen Dependencies hat, ist er als **reiner Unit-Test** prüfbar:

- xUnit-Tests gegen die 2 README-Beispiele (siehe README §1.2: "You get 2 examples to test your application.")
- Edge-Cases: unlösbar, mehrdeutig, Single-Cell-Cage (Sum=N erzwingt Wert N)
- Performance-Boundary: < 2 Sekunden (UC11 AC11.2)

Use-Case-Anker

- UC11 — Auto Solve
- [ADR-010](#)
- [ADR-013](#)

§8.6 Datenzugriff

Konzept

Datenzugriff über **Entity Framework Core 10** mit eigenem `SudokuDbContext` . Das Schema wird über **EF-Migrations** versioniert (Source of Truth liegt in `KillerSudoku.Data/Persistence/Migrations/`). Aus den Migrations wird das von README §1.3 geforderte deklarative Submission-Skript **Datenbank-Skript** per `dotnet ef migrations script` exportiert — beides bleibt konsistent.

DbContext-Scope

Aspekt	Lösung
Lebenszyklus	Scoped pro Blazor-Server-Circuit (eine Connection pro Tab/Session)
Connection-String	Aus <code>appsettings.json</code> (siehe Kapitel 7 §7.3)
Tracking	Default <code>AsTracking</code> für UPDATES (GameCell), <code>AsNoTracking</code> für Read-Only-Queries (Puzzle-Liste, Highscore)
Transaktionen	Pro Service-Call eine Transaction wenn mehrere Tabellen geschrieben werden (z.B. UC05: Puzzle + Cage + CageCell)

Migrations

Migration-Strategie: **EF-Migrations als Source of Truth, Datenbank-Skript** als generiertes Submission-Artefakt.

Workflow:

1. Schema-Änderung in den C#-Entities oder im `SudokuDbContext` → `dotnet ef migrations add <Name>` erzeugt eine neue Migration unter `KillerSudoku.Data/Persistence/Migrations/`.
2. Beim Container-Start (oder via `dotnet ef database update`) werden alle ausstehenden Migrations angewendet.
3. Das deklarative Submission-Skript **Datenbank-Skript** wird per `dotnet ef migrations script --idempotent -o db/sudoku.sql` exportiert — damit hat der Prüfer ein eigenständiges, idempotentes Skript zum manuellen Ausführen (README §1.3).

Begründung:

- EF-Migrations geben versioniertes, reversibles Schema-Management (`dotnet ef migrations remove` / `database update <vorherige>`)
- Der Export liefert genau das von §1.3 geforderte **Datenbank-Skript** ohne manuelle Pflege zweier Wahrheits-Sources
- Idempotenz-Modus (`--idempotent`) macht das Skript wiederholt ausführbar für SSMS / `sqlcmd`

EF-Modell-Mapping

C#-Klasse	Tabelle	Bemerkung
<code>AppUser</code>	<code>AppUser</code>	Spalten-Name aus T-SQL-Reserved-Konflikt — siehe ADR-005
<code>Puzzle</code>	<code>Puzzle</code>	KEINE Solution-Spalte (siehe ADR-007)
<code>Cage</code>	<code>Cage</code>	mit <code>[Sum]</code> -Property (T-SQL-Reserved)
<code>CageCell</code>	<code>CageCell</code>	<code>RowIndex</code> / <code>ColIdx</code> statt <code>Row</code> / <code>Col</code> — siehe ADR-006
<code>Game</code>	<code>Game</code>	Pause-Felder siehe ADR-009
<code>GameCell</code>	<code>GameCell</code>	<code>CellValue</code> NULL-fähig
<code>PencilMark</code>	<code>PencilMark</code>	eigene Tabelle, kein JSON — siehe ADR-008
<code>HintLog</code>	<code>HintLog</code>	Audit-Log

Highscore-View

Highscore ist eine **DB-View** `vw_Highscore` (siehe **Datenbank-Skript** und [ADR-003](#)). EF mappt sie als read-only Keyless-Entity. Der Service `IHighscoreService.GetTopAsync` selektiert nur aus dieser View und sortiert nach `Score DESC`.

Anker

- **ER-Modell** — vollständiges ERD mit Spalten-Typen
- **Datenbank-Skript** — Schema
- **Funktionalitäts-Matrix** — Service × DB-Operations-Matrix

§8.7 UI-Komponenten

Konzept

Die UI ist in einer **Blazor-Component-Hierarchie** organisiert. Pro Screen (S1–S7) existiert eine Root-Component (`@page` -Route), die kleinere Children komponiert. Komponenten sind nach **Single Responsibility** geschnitten und einzeln via bUnit testbar.

Component-Hierarchie

Übersicht aus **Funktionalitäts-Matrix** (Spalte "Blazor-Component"):

```
Layout.razor
├── Home.razor (S1, public, UC01)
│   ├── RulesPanel.razor
│   │   └── MiniSudokuExample.razor
│   ├── Register.razor (S2, public, UC02)
│   │   └── RegisterForm.razor
│   ├── Login.razor (S3, public, UC03)
│   │   └── LoginForm.razor
│   ├── Puzzles.razor (S4, [Authorize], UC12)
│   │   ├── FilterBar.razor
│   │   └── PuzzleCard.razor
│   ├── EnterPuzzle.razor (S5, [Authorize], UC04+UC05)
│   │   ├── PuzzleGrid.razor
│   │   └── CageEditor.razor
│   ├── PlayPuzzle.razor (S6, [Authorize], UC06–UC09, UC13, UC14)
│   │   ├── PuzzleGrid.razor
│   │   │   └── PencilMarkLayer.razor (UC14)
│   │   ├── Toolbar.razor
│   │   │   ├── HintButton.razor (UC07)
│   │   │   ├── PauseButton.razor (UC13)
│   │   │   └── CheckSolutionButton.razor (UC09)
│   └── Highscore.razor (S7, [Authorize], UC08)
│       └── HighscoreTable.razor
```

Quell-Anker

- Screen-Inventar: **Funktionalitäts-Matrix**
- Component × UC × Service × DB: **Funktionalitäts-Matrix**
- Visuelle Briefings: **Mockup-Briefings**

Component-Kommunikation

Pattern	Verwendung
[Parameter]	Eltern → Kind (Down)
EventCallback<T>	Kind → Eltern (Up)
CascadingValue	Spielzustand in <code>PlayPuzzle</code> -Subtree
Service-Injection via DI	Components → Service-Layer

State-Management

Spielzustand (Grid, Pencil Marks, Pause-Status) lebt auf dem Server pro Blazor-Circuit. Bei Disconnect/Reconnect lädt der Component-Init aus `IGameService` den persistierten Zustand aus der DB nach (Auto-Save bei jedem Move — siehe **UC13** "Browser-Tab schliessen ohne Pause → State wird trotzdem persistiert").

§8.8 Internationalisierung

Konzept

UI-Sprache: Deutsch (einsprachig). Es ist keine Internationalisierung (i18n) implementiert.

Begründung

- Das README spezifiziert keine i18n-Pflicht (kein "must" oder "required" zu Sprachen)
- Der Wettbewerb ist im deutschsprachigen Raum verortet (Skills Battle)
- Die Stakeholder-Tabelle in [Kapitel 1 §1.3](#) hält fest: "UI-Sprache deutsch."

Umsetzung

- Alle UI-Strings sind deutsch hardkodiert in Razor-Markup oder als `const string` (kein `LocalizedString`)
- Fehlermeldungen sind deutsch (Beispiele aus **Validation-Regeln**):
- "Schwierigkeit muss 1, 2 oder 3 sein."
- "Username oder Passwort falsch."
- "Puzzle ist nicht lösbar."
- Datums-/Zeit-Formate: `de-CH` Culture (kann via `appsettings.json` umgestellt werden, ist aber kein Pflicht-Feature)

Falls i18n nachträglich gefordert wird, ist die Migration unproblematisch (`.resx` -Files +

`@inject IStringLocalizer`). Aktuell aus Scope ausgeschlossen.

§8.9 Sicherheit

Konzept

Sicherheit ist als **Defense in Depth** über mehrere Layer organisiert. Die hier aufgeführten Massnahmen leiten sich direkt aus den README-Vorgaben (Login Pflicht, Solution-Eindeutigkeit) und den Validation-Regeln V01–V16 ab.

Massnahmen-Matrix

Bedrohung	Massnahme	Layer	Anker
Account-Diebstahl (Replay/Theft)	Auth-Cookie: <code>HttpOnly</code> + <code>Secure</code> + <code>SameSite=Lax</code>	Server (Identity-Konfiguration)	V15 , UC03 AC03.3
Cross-Site Request Forgery	ASP.NET Antiforgery Token (Standard in Blazor Server <code>EditForm</code>)	Server	V15
Cross-Site Scripting (XSS)	Razor <code>@expression</code> → automatisches HTML-Encoding; kein <code>@((MarkupString))</code> für User-Content	Server	V14
SQL-Injection	EF Core Parameterized Queries (keine String-Konkatenation in SQL)	Server	implicit (EF Core Default)
Passwort-Klartext-Leak	PBKDF2-Hash via ASP.NET Identity; NIE Klartext loggen	Server	V03 , §8.4
Account-Enumeration	Generische Login-Fehlermeldung ("Username oder Passwort falsch")	Server	V03 , UC03 AC03.1
Brute-Force-Login	Rate-Limit: 5 Versuche / 5 Minuten	Server	V04
Cross-User-Mutation	Server-side Check <code>game.UserId == currentUserId</code> vor jedem Write	Server	V16
Unauthorized Routes	<code>[Authorize]</code> -Attribut + Redirect zu <code>/login</code>	Server	V16
Puzzle-Manipulation (Hash-Probleme bei Solve)	Solver-Logik ist serverseitig, kein Trust auf Client-Input	Server	UC09

Cookie-Konfiguration

```
services.ConfigureApplicationCookie(options =>
{
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.ExpireTimeSpan = TimeSpan.FromHours(2);
    options.LoginPath = "/login";
    options.AccessDeniedPath = "/access-denied";
});
```

Cookie-Lifetime 2 h fix (kein SlidingExpiration): minimiert das Theft-Window bei kompromittiertem Cookie. Für eine Login-Required-App ohne sensitive Long-Running-Sessions die sicherere Wahl gegenüber 24 h mit Sliding-Refresh. **SameSite=Lax** (statt **Strict**) erlaubt Top-Level-Navigation mit Cookie (z.B. Klick auf externen Link, der zurück führt). **Lax** ist OWASP-Recommendation für Web-Apps mit normalem Login-Flow.

Verweise

- **Use-Cases-Dokument** — UC01–UC14
- **Validation-Regeln** — V01–V16
- **Funktionalitäts-Matrix** — Service- & UI-Inventar
- **ER-Modell** — DB-Modell & Design-Entscheidungen
- [Kapitel 7 — Verteilungssicht](#)
- [Kapitel 9 — Architekturentscheidungen](#)

9 Architekturentscheidungen (ADRs)

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Dieses Kapitel dokumentiert die wesentlichen Architekturentscheidungen als ADRs (Architecture Decision Records). Jeder Eintrag folgt dem Format:

- **Titel** — Was wurde entschieden
- **Status** — `Akzeptiert` (alle hier dokumentierten ADRs sind angenommen)
- **Kontext** — Welches Problem / welche Anforderung führt zur Entscheidung
- **Entscheidung** — Was wurde konkret gewählt
- **Konsequenz** — Welche positiven & negativen Folgen ergeben sich

ADR-001 — Stack: .NET 10 + Blazor Server + MS-SQL Server

Status: Akzeptiert

Kontext: Die Aufgabenstellung (Skills Battle 2026 — Skill 7) erlaubt verschiedene Implementations-Stacks (PHP, Node.js, .NET). README §1.3 fordert lediglich: "create a database named 'sudoku' on **MySQL or MS-SQL** server running on your machine." Die UI-Technologie ist frei wählbar.

Entscheidung: Stack: **.NET 10** (Blazor Server) als Anwendungs-Framework und **MS-SQL Server Express** als Datenbank.

Konsequenz:

- (+) Strenges Typsystem (C# 13) reduziert Fehler bei Solver- und Domain-Logik
- (+) Blazor Server: vollständiges C#-Stack (Frontend + Backend), keine Sprach-Wechsel zwischen Layern
- (+) ASP.NET Core Identity bietet hardened-Auth out-of-the-box (siehe §8.1)
- (+) EF Core 10 + MS-SQL Express: konsistentes Tooling, Trusted Connection für lokales Setup
- (+) bUnit-Component-Tests + xUnit Unit-Tests sind im selben Test-Projekt möglich
- (–) Blazor Server benötigt permanente SignalR-Verbindung (siehe §7.1) — bei Disconnect Verlust des UI-State (mitigiert via Auto-Save in DB, **UC13**)
- (–) .NET-Setup auf Prüfer-Laptop muss vorhanden sein (Mitigation: Submission liefert Build-Output in `bin/Release/net10.0/`, siehe §7.4)

ADR-002 — Cage-Modellierung relational (Cage + CageCell)

Status: Akzeptiert

Kontext: Ein Killer-Sudoku besteht aus Cages: Gruppen zusammenhängender Zellen mit einer Soll-Summe. Pro Puzzle gibt es 1–N Cages, jeder Cage hat 1–9 Zellen. Eine Zelle gehört genau einem Cage an. Modellierungs-Optionen:

1. **Relational** — Tabelle `Cage` (Id, PuzzleId, Sum) + Junction-Tabelle `CageCell` (CageId, RowIdx, ColIdx)
2. **JSON in Puzzle** — Eine Spalte `CagesJson NVARCHAR(MAX)` mit Array von Cages

Entscheidung: Variante 1 (relational). Siehe Schema in **Datenbank-Skript** (Zeilen 67–96).

Konsequenz:

- (+) FK-Integrität & DB-CHECK-Constraints ($\text{Sum} \in [1, 45]$, $\text{Row/Col} \in [0, 8]$) — siehe **V06**
- (+) Einfache SQL-Queries für Cage-Sum-Validation (UC09) — siehe Sample-Queries in **ER-Modell**
- (+) Indexierbarkeit (Index `IX_Cage_PuzzleId`)
- (+) Trigger `trg_CageCell_UniquePerPuzzle` lässt sich nur in der relationalen Variante sinnvoll umsetzen
- (–) Mehr Joins beim Lesen eines Puzzles (zwei Queries: Cages + CageCells, oder ein Join)
- (–) Speicher-Overhead minimal (TINYINT-Spalten + Composite PKs)

ADR-003 — Highscore als View statt Tabelle

Status: Akzeptiert

Kontext: UC08 (Show High Score) zeigt die Top-N abgeschlossenen Games sortiert nach Score. Optionen:

1. **Denormalisierte Tabelle** `Highscore` mit Sync-Logik bei UC10 (Save Result)
2. **View** `vw_Highscore` über `Game JOIN AppUser JOIN Puzzle`

Entscheidung: Variante 2 (View). Siehe **Datenbank-Skript** Zeilen 214–232.

Konsequenz:

- (+) Single Source of Truth: kein Sync-Risiko zwischen `Game` und `Highscore`
- (+) Automatisch konsistent bei UPDATE auf `Game`
- (+) `WHERE g.IsCompleted = 1 AND g.Score IS NOT NULL` stellt sicher, dass nur abgeschlossene Spiele erscheinen — passt zu UC10 AC10.3
- (–) Performance: bei sehr vielen Games ($>10^6$) wäre View langsamer als materialisierte Tabelle. Für die Wettbewerbs-Submission (lokales Setup, wenige Spiele) irrelevant.

- (–) Pagination der View erfordert `OFFSET/FETCH` in jeder Abfrage statt vorberechneter Sortierung
- **Implementierungs-Realität:** Der `HighscoreService` liest aktuell **nicht** über die View, sondern direkt über LINQ-Joins (`_db.Games ⋈ AppUser ⋈ Puzzle WHERE IsCompleted = 1`). Der `SudokuDbContext` mappt `vw_Highscore` nicht als Keyless-Entity. Die View bleibt im SQL-Schema als Reserve — ein späterer Wechsel auf `FromSqlRaw("SELECT ... FROM vw_Highscore")` oder einen Keyless-DbSet ist möglich, ohne dass das Schema geändert werden muss.
- Trade-Off-Begründung: LINQ erlaubt typsischeren Difficulty-Filter (`GetTopAsync(limit, byte? difficulty)`), Order-Stabilität (`OrderByDescending(Score).ThenBy(TimeSeconds)`) und Rank-Vergabe direkt im C#-Code. Bei wachsender Datenmenge oder Bedarf an View-Wartbarkeit ist Re-Mapping auf `vw_Highscore` der nächste sinnvolle Schritt.

ADR-004 — "Eine Zelle pro Cage pro Puzzle" via Trigger

Status: Akzeptiert

Kontext: Die Regel "jede Zelle gehört zu **genau einem** Cage pro Puzzle" (siehe UC04 AC04.2 und **V06**) lässt sich auf Datenbank-Ebene nicht direkt mit einer einzelnen UNIQUE-Constraint ausdrücken, weil `CageCell` kein direktes `PuzzleId`-Feld hat (PuzzleId steckt in `Cage`).

Optionen:

1. **Composite FK über (PuzzleId, RowIdx, ColIdx)** — würde `PuzzleId` zusätzlich in `CageCell` duplizieren
2. **AFTER INSERT/UPDATE Trigger** — prüft Cross-Cage-Konflikt im Puzzle
3. **Nur Application-Layer-Validierung** — kein DB-Schutz

Entscheidung: Variante 2 (Trigger). Konkret `trg_CageCell_UniquePerPuzzle` (siehe **Datenbank-Skript** Zeilen 100–121):

```

CREATE OR ALTER TRIGGER trg_CageCell_UniquePerPuzzle
ON CageCell
AFTER INSERT, UPDATE
AS
BEGIN
    ...
    IF EXISTS (
        SELECT 1
        FROM CageCell cc
        JOIN Cage c1 ON c1.Id = cc.CageId
        JOIN inserted i ON i.RowIdx = cc.RowIdx AND i.ColIdx = cc.ColIdx
        JOIN Cage c2 ON c2.Id = i.CageId
        WHERE c1.PuzzleId = c2.PuzzleId
        AND cc.CageId <> i.CageId
    )
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50001, 'CageCell: Cell already assigned to another cage in the same puzzle.', 1
    END
END;

```

Konsequenz:

- (+) Defense-in-Depth: Application-Validierung (V06) + DB-Trigger
- (+) Schema-Normalisierung erhalten — kein redundantes `PuzzleId` in `CageCell`
- (+) Klare Fehlermeldung im Konflikt-Fall (`THROW 50001`)
- (–) Trigger-Logik ist weniger sichtbar als deklarative Constraints
- (–) Performance-Overhead bei Bulk-Inserts (eher irrelevant: 81 Zellen pro Puzzle)
- Begründung im Kommentar des SQL-Skripts (Zeile 84–86): "Constraint 'jede Zelle pro Puzzle in genau einem Cage' wird über einen INSTEAD-OF-Trigger durchgesetzt"

Anmerkung zum SQL-Kommentar: Der Kommentar im Skript spricht von "INSTEAD-OF-Trigger", umgesetzt wird tatsächlich ein `AFTER INSERT, UPDATE` Trigger. Funktional identisch (Rollback bei Konflikt), aber Wording im Kommentar bleibt zur Konsistenz mit dem Submission-File.

Anmerkung zum EF-Modell: Der Trigger `trg_CageCell_UniquePerPuzzle` lebt ausschließlich im SQL-Skript `db/sudoku.sql`. Er ist im `SudokuDbContext` (`OnModelCreating`) **nicht** als `HasTrigger`-Konfiguration oder Migration-Operation modelliert. Konsequenz: Wer die Datenbank ausschließlich über `EnsureCreated` oder eine künftige EF-Migration aufsetzt, würde den Trigger nicht erhalten. Die aktuelle Submission setzt das Schema immer über **Datenbank-Skript** (Container-Entrypoint oder manueller `sqlcmd` -Aufruf), daher ist der Trigger im Lauf garantiert.

ADR-005 — AppUser statt User (T-SQL Reserved Keyword)

Status: Akzeptiert

Kontext: Die Benutzer-Tabelle würde idiomatisch `User` heissen. In T-SQL ist `USER` jedoch ein reserviertes Keyword (für `USER_NAME` etc.), was sowohl im Schema-Skript als auch in EF-LINQ-Queries ständige Eckige-Klammern-Quotation erfordern würde.

Entscheidung: Tabelle und C#-Entity heissen `AppUser`. Siehe **ER-Modell** und **Datenbank-Skript** Zeilen 36–48.

Konsequenz:

- (+) Kein Quoting notwendig, klare Lesbarkeit in SQL und C#
 - (+) Konvention findet sich in vielen ASP.NET-Identity-Templates (`AspNetUsers`, `ApplicationUser` etc.)
 - (–) Leichte Reibung mit dem Use-Case-Vokabular ("User" als Begriff bleibt im Sprachgebrauch)
-

ADR-006 — RowIdx / ColIdx statt Row / Col

Status: Akzeptiert

Kontext: Sudoku-Zellen werden durch Zeilen- und Spalten-Index identifiziert. Idiomatisch wären die Spalten-Namen `Row` und `Col`. T-SQL reserviert allerdings `ROW` (z.B. in `ROW_NUMBER` und Table-Hints).

Entscheidung: Spalten heissen `RowIdx` und `ColIdx` (TINYINT, Range 0–8). Siehe **ER-Modell** und **Datenbank-Skript**.

Konsequenz:

- (+) Kein Quoting in SQL
 - (+) Suffix `Idx` macht den Zero-Based-Index explizit (nicht 1-Based wie in der UI)
 - (–) Etwas länger als `Row / Col` — minimaler Cosmetic-Impact
-

ADR-007 — KEINE Solution-Spalte in Puzzle

Status: Akzeptiert

Kontext: Ein Puzzle hat genau eine eindeutige Lösung (gefordert durch README §1 "The solution **must be unique**." und durchgesetzt durch UC05). Naheliegende Optimierung wäre, die berechnete Lösung beim Save zu cachen.

Die Aufgabenstellung verbietet das jedoch explizit. README §2.1 UC4 wörtlich:

"No solution is recorded. Solutions must be calculated with an algorithm."

Entscheidung: Die Tabelle `Puzzle` enthält **keine** Solution-Spalte. Lösungen werden bei jedem Bedarf (UC05, UC07, UC09) on-the-fly via `ISolverService.Solve` berechnet.

Konsequenz:

- (+) Konform zur strikten README-Vorgabe — bestehensrelevant
- (+) Single Source of Truth: Solver ist die Autorität, kein DB-Cache kann veralten
- (+) Test: SQL-Query auf `INFORMATION_SCHEMA.COLUMNS` zeigt keine `Solution`-Spalte (UC04 AC04.4)
- (–) Solver wird mehrfach aufgerufen pro Game-Session (z.B. mehrere Hints) — mitigiert durch In-Memory-Cache pro Circuit, **niemals** persistent
- Konsequenz aus Strict-Word-Audit S4 in **Use-Cases-Dokument**

ADR-008 — Pencil Marks als eigene Tabelle (statt JSON in GameCell)

Status: Akzeptiert

Kontext: UC14 (Pencil Marks) erlaubt 0–9 Kandidaten-Markierungen pro Zelle. Modellierungs-Optionen:

1. **Eigene Tabelle** `PencilMark` mit Composite PK (`GameId`, `RowIdx`, `ColIdx`, `MarkValue`)
2. **JSON-Spalte** in `GameCell` (z.B. `PencilMarksJson NVARCHAR(50)`)
3. **Bitmaske** in `GameCell` (z.B. `TINYINT PencilBitmask` — Bits 0–8 für Zahlen 1–9)

Entscheidung: Variante 1 (eigene Tabelle). Siehe **ER-Modell** und **Datenbank-Skript** Zeilen 180–192.

Konsequenz:

- (+) Konsistenz mit relationaler Cage-Modellierung (ADR-002)
- (+) CHECK-Constraint `CK_PencilMark_Value` (Range 1–9) auf DB-Ebene
- (+) Composite PK verhindert Duplikate (siehe **V13**)
- (+) Indexier-/Query-bar (z.B. "alle Marks für Gameld X")
- (–) Mehr Rows in DB (worst case $81 \times 9 = 729$ Marks pro Game) — bei lokalem Setup unkritisch
- (–) JSON wäre kompakter, aber kein Schutz vor Invariant-Verletzungen

ADR-009 — Pause via TotalPausedSeconds + PausedAt

Status: Akzeptiert

Kontext: UC13 (Pause & Resume) erfordert, dass die gespielte Netto-Zeit korrekt berechnet wird, auch wenn der User mehrmals pausiert. Optionen:

1. **Multi-Row Pause-Log** — Separate Tabelle `PauseEvent (GameId, PausedAt, ResumedAt)`, Summe aller Differenzen ist die Pause-Zeit
2. **Single Row mit Accumulator** — Felder `IsPaused`, `PausedAt`, `TotalPausedSeconds` direkt in `Game`
3. **Nur StartTime adjustieren** — Bei Resume die `StartTime` nach hinten schieben

Entscheidung: Variante 2. `Game` hat drei Pause-Felder:

- `IsPaused BIT` — aktuell pausiert ja/nein
- `PausedAt DATETIME2 NULL` — Zeitpunkt der aktuellen Pause
- `TotalPausedSeconds INT` — kumulierte Pause-Zeit über alle bisherigen Intervalle

Berechnung beim Game-Ende:

```
TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds
```

Siehe **Datenbank-Skript** Zeilen 128–149 und **ER-Modell**.

Konsequenz:

- (+) Einfach zu queryen, kein Aggregat über Sub-Tabelle nötig
- (+) Audit-Spur durch `PausedAt` (letzter Pause-Beginn) — operative Sichtbarkeit
- (+) Konsistenz-Check `CK_Game_TotalPaused CHECK (TotalPausedSeconds >= 0)` (**V12**)
- (–) Keine Historie aller Pause-Events (z.B. "wie oft wurde pausiert"). Aus README nicht gefordert.
- (–) Resume-Logik muss atomic sein (read `PausedAt`, compute Δ , add to `TotalPausedSeconds`, set `IsPaused=0`, `PausedAt=NULL`) — wird via Service-Transaction abgesichert

ADR-010 — Solver-Output: Limit auf 2 Lösungen

Status: Akzeptiert

Kontext: Der Solver muss drei Fälle unterscheiden (siehe **V07**):

- 0 Lösungen → "Puzzle ist nicht lösbar"
- 1 Lösung → speichern (UC05) bzw. Hint geben (UC07)
- ≥ 2 Lösungen → "Lösung muss eindeutig sein"

Wenn der Solver mehrere Lösungen finden würde, könnte er stundenlang laufen (10^9 + Möglichkeiten bei wenig vorgegebenen Zellen).

Entscheidung: Der Solver-Algorithmus bricht **nach der zweiten** gefundenen Lösung ab.
Rückgabe:

```
public record SolveResult(int Solutions, int[,]? Solution);
// Solutions ∈ {0, 1, 2}
// 2 bedeutet: "mindestens 2 gefunden, weitere möglich"
```

Siehe UC11 Main Flow Schritt 5:

"Bei Solutions $\geq 2 \rightarrow$ nicht eindeutig (Solver bricht nach 2. gefundener Lösung ab)"

Konsequenz:

- (+) Performance: Worst-Case bounded, keine Abbruch-Heuristik nötig
- (+) Genügt für UC05-Entscheidung (Save ja/nein) und UC11-Aussage (Eindeutigkeit ja/nein)
- (+) Boundary-Test: UC11 AC11.2 "< 2 Sekunden"
- (–) Keine genaue Anzahl Lösungen bei ambiguous Puzzles — aber: nicht gefordert
- (–) Hint-Algorithmus muss bei `Solutions != 1` graceful failen (Hint nur bei eindeutiger Lösung sinnvoll)

ADR-011 — Test-Pyramide: xUnit + bUnit + WebApplicationFactory + Playwright .NET

Status: Akzeptiert

Kontext: README §3.2 fordert wörtlich:

"Running the test cases should be implemented with a **test framework** and is part of the submission (e.g. JUnit in Java)."

Plus §3.1 (mind. 1 positiver, 1 negativer, ggf. 1 Boundary-Test pro UC).

Test-Pyramide-Optionen für den .NET-Stack:

Layer	Framework-Optionen
Unit	xUnit, NUnit, MSTest
Component (Razor)	bUnit, AngleSharp-DOM-Setup
Integration	xUnit + <code>WebApplicationFactory<T></code> (offiziell)
E2E	Playwright .NET, Selenium, Cypress

Entscheidung:

- **Unit:** xUnit (Solver, Service-Methoden, Validation-Regeln)
- **Component:** bUnit (Razor-Components — Render, Event, State)
- **Integration:** xUnit + `WebApplicationFactory` (echte DB-Roundtrips, Auth-Flow)
- **E2E:** Playwright .NET (kritische User-Flows: Register → Login → Solve)

Konsequenz:

- (+) Alle Layer als `dotnet test` aufrufbar — eine CI-Pipeline
- (+) bUnit ist der De-facto-Standard für Blazor-Components (Microsoft-empfohlen)
- (+) Playwright .NET liefert auch UI-Test-Recording (Trace-Files)
- (+) Solver-Tests als reine Unit-Tests möglich, da Solver Pure Function (§8.5)
- (–) Vier Frameworks im Stack — Lernkurve für Prüfer, der das Test-Protokoll liest. Mitigation: Test-Protokoll listet Framework pro Test-Case in der CSV (**Test-Protokoll**).
- (–) Playwright benötigt Browser-Binary-Download beim ersten Run

ADR-012 — Score-Formel: $\max(0, 10000 - \text{TimeSeconds} - \text{HintsUsed} \times 300)$

Status: Akzeptiert

Kontext: README §2.1 UC8 fordert: "Implement rules for a high score based on the time needed and the number of hints used." Die konkrete Formel ist frei wählbar.

Anforderungen an die Formel:

- Schnellere Spiele → höherer Score
- Mehr Hints → niedrigerer Score (lt. UC07: "should be considered for the high score")
- Keine negativen Scores
- Ganzzahlig

Entscheidung:

```
Score = max(0, 10000 - TimeSeconds - HintsUsed × 300)
```

Siehe UC08 Score-Formel:

- Begründung: Zeit-Penalty linear (1 Pkt/Sek); Hints kosten 5 Min Äquivalent
- Floor bei 0 → keine negativen Scores

Konsequenz:

- (+) Einfach nachvollziehbar — Prüfer kann manuell prüfen
- (+) 5-Min-Penalty pro Hint ist eine "schmerzhaft" aber nicht ruinöse Wertung
- (+) `max(0, ...)` verhindert negative Scores → DB-Constraint `CK_Game_Score CHECK (Score >= 0)` erfüllt (**V12**)
- (–) Theoretisch könnte ein User in 9 999 Sekunden ohne Hints noch 1 Pkt erreichen — aber: nicht negativ, akzeptabel
- Test-Anker: **AC08.1** (Score ganzzahlig nach Formel)

ADR-013 — Hint-Strategie: Naked-Single → Cage-Forced → Solver-Fallback

Status: Akzeptiert

Kontext: UC07 (Ask for a Hint) verlangt einen Hint, der dem Spieler hilft. README:

"Develop a **suggestion** for the hint, document it, and implement your solution."

Es gibt mehrere Hint-Qualitäten:

1. **Naked Single** — Zelle, in der nur 1 Wert constraint-konform möglich ist ("Logikhint")
2. **Cage-Forced** — Cage, dessen Belegung eindeutig durch die Summe + Constraints forciert ist
3. **Solver-Fallback** — wenn kein Logik-Hint möglich: einfach eine korrekte leere Zelle füllen

Entscheidung: Hint-Algorithmus versucht in dieser Reihenfolge:

1. **Strategy A — Naked Single:** Suche eine leere Zelle, deren mögliche Werte nach Anwendung aller Constraints (Row/Col/Nonet/Cage) genau eine Zahl ergibt. Wenn gefunden: Hint enthält "Hier kann nur N stehen".
2. **Strategy B — Cage-Forced:** Wenn keine Naked-Single existiert, suche einen Cage, dessen Lösung eindeutig ist (z.B. 2-Zellen-Cage mit Sum=17 in unterschiedlichen Rows → muss `{8,9}` sein).
3. **Fallback:** Wenn beide Strategien scheitern (selten bei harten Puzzles), lass den Solver eine beliebige leere Zelle der bekannten Lösung füllen.

Siehe UC07 Hint-Vorschlag (dokumentiert).

Konsequenz:

- (+) Pädagogischer Mehrwert: Hint erklärt die Logik (Strategy A/B), nicht nur das Ergebnis
- (+) Solver wird nur als Fallback gebraucht → Performance besser bei einfachen Puzzles
- (+) HintsUsed-Counter wird in jedem Fall inkrementiert (siehe UC07 Main)
- (–) Hint-Algorithmus ist komplexer als ein reiner Solver-Aufruf — mehr Code, mehr Tests
- (–) Tests müssen alle drei Pfade abdecken (Strategy A, B, Fallback)
- Test-Anker: **AC07.1** (Hint füllt nur korrekte Zelle)

ADR-014 — Auswahl der 3 zusätzlichen Use Cases

Status: Akzeptiert

Kontext: README §2.3 fordert:

"Add additional use cases to the use case diagram. Think of **3 additional** use cases that you can implement. Choose them based on the existing entries. Write down your decision in the documentation."

Kandidaten für zusätzliche UCs:

- Browse / Filter Puzzles
- Pause / Resume Game
- Pencil Marks (Candidate Notes)
- Edit Profile / Change Password
- Delete eigenes Puzzle
- Statistik-Dashboard
- Multiplayer / Live-Wettkampf
- Daily-Challenge

Entscheidung: Gewählte UCs (siehe **Use-Cases-Dokument**):

- **UC12 — Browse / Filter Puzzles** — Praktischer UX-Mehrwert, DB-Query-relevant (Integration-Test-Anker)
- **UC13 — Pause / Resume Game** — Reale User-Anforderung (Sudoku-Spiele dauern lange), persistiert Game-State (Component- und Integration-Test-Anker)
- **UC14 — Pencil Marks** — Standard-Feature aller Sudoku-Apps, demonstriert UI-State-Management (Component-Test-Anker)

Begründung der Auswahl (wörtlich aus Use-Cases-Dokument):

- UC12 Browse/Filter — Praktischer Mehrwert (UX), DB-Query-relevant (Integration-Tests)
- UC13 Pause/Resume — Reale User-Anforderung (Sudoku-Spiele dauern lange), persistiert Game-State (Component + Integration)
- UC14 Pencil Marks — Standard-Feature aller Sudoku-Apps, demonstriert UI-State-Management (Component-Test)

→ Die 3 UCs decken bewusst 3 verschiedene Test-Schwerpunkte ab.

Konsequenz:

- (+) Jeder UC zielt auf eine andere Test-Schicht (DB-Integration / Game-State / UI-Component) — gute Demonstration der Test-Pyramide aus [ADR-011](#)
- (+) Jeder UC ergänzt Kernfunktionen (Solve-Flow), ohne neue Personas einzuführen → Scope bleibt eng
- (+) Pencil Marks und Pause sind Standard-Feature-Erwartungen → User-Experience-Boost
- (–) Bewusst NICHT gewählt: Multiplayer (würde Auth- und Sync-Komplexität explodieren), Admin (zweite Persona, mehr Routen)
- (–) Browse-Filter erfordert Pagination-Logik (siehe UC12 AC12.2)

ADR-015 — Single-Image Container für Prüferbequemlichkeit

Status: Akzeptiert

Kontext: Die README-Aufgabenstellung verlangt explizit: "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine**" (§1.3). Das bedeutet, jeder Prüfer muss lokal eine MS-SQL-Instanz installieren, das Schema deployen, die App konfigurieren und starten. Diese Reibung ist für die Bewertung nicht förderlich.

Alternativen:

1. **Native-Only (Standard).** Prüfer installiert MS-SQL Express + .NET 10 SDK manuell, führt **Datenbank-Skript** aus, startet App via `dotnet run`. Standard und gut dokumentiert (siehe [§7.2 / §7.3](#)) — aber: Setup-Reibung, Plattform-Drift macOS/Windows/Linux.
2. **Klassisches Docker-Compose mit 2 Services.** Ein `mssql`-Service + ein `app`-Service mit Network-Bridge. Best-Practice für Production (saubere Trennung, Skalierbarkeit) — aber: zwei Container, zwei Lifecycle-Probleme, Wait-for-DB-Race-Condition vor App-Start.
3. **Single-Image (DB + App in einem Container).** Multi-Stage Build mit `mssql/server:2022-latest` als Base + .NET-Runtime als Layer + Supervisor-Script (`docker-entrypoint.sh`) für sqlservr + dotnet. Anti-Pattern für Production, aber **ein einziger `docker run`-Befehl** für den Prüfer.

Entscheidung: Option 3 (Single-Image) wird als primärer Container-Pfad ausgeliefert, parallel zur nativen Variante.

Begründung:

- Der Wettbewerbs-Kontext ist Submission, nicht Production. Der Prüfer bewertet ein Mal — Skalierung oder Update-Pfade sind nicht relevant.
- README §1.3 ("running on your machine") wird trotz Container erfüllt — der Container läuft auf der Maschine des Prüfers, die DB läuft im Container des Prüfers. Kein Cloud-Service involviert.
- Das Image wird bei jedem `main`-Push via `.github/workflows/deploy.yml` gebaut und nach `ghcr.io/tim-fischer-zh/killer-sudoku:latest` gepusht → Prüfer kann mit einem `docker pull` ohne Local-Build verifizieren.
- Supervisor-Script handhabt Race-Condition (Wait-for-DB), idempotenter Schema-Apply via Marker-File.
- Die Native-Variante bleibt als Fallback dokumentiert für Prüfer ohne Docker.

Konsequenz:

- (+) `docker run -p 8080:8080 ghcr.io/tim-fischer-zh/killer-sudoku:latest` und Browser → fertig.
- (+) Reproduzierbarkeit garantiert (Image-SHA in GHA-Tags).
- (+) MS-SQL EULA wird im Image akzeptiert (via `ACCEPT_EULA=Y`) — kein manuelles Setup.
- (–) Anti-Pattern für Production: zwei Prozesse in einem Container (Single-Process-Per-Container ist Best Practice).
- (–) Bigger image footprint (~ 1.5 GB inkl. mssql-Layer).
- (–) Logs der zwei Prozesse vermischen sich (stdout). Akzeptabel für Submission-Kontext.
- (–) Healthcheck deckt nur die App ab (`curl :8080/health`), nicht die DB direkt — DB-Health wird implizit über App-Health geprüft.

Operatives Setup:

- `Dockerfile`, `docker-entripoint.sh`, `docker-compose.yml`, `docker-compose.prod.yml`, `.dockerignore` im Repo-Root
- GitHub-Actions-Workflow für Build & Push nach `ghcr.io` sowie Production-Deploy (siehe ADR-016)
- Persistenz via Named-Volume `mssql-data`
- Connection-String über `ConnectionStrings__Sudoku` Env-Var override-bar

→ Vollständiges Setup in [§7.5 Container-Deployment](#).

ADR-016 — Production-Deployment via Self-Hosted Runner + Cloudflare-Tunnel

Status: Akzeptiert

Kontext: Über das von der Aufgabenstellung geforderte lokale Setup hinaus soll die App **zusätzlich** als Live-Demo unter einer öffentlichen Domain (`web17skill.com`) erreichbar sein, damit der Prüfer eine bereits laufende Instanz aufrufen kann, ohne lokal etwas installieren zu müssen. Diese Production-Variante ist **nicht** Pflicht laut Spec — sie ist ein Bonus.

Zu entscheiden waren zwei Achsen:

1. **Deploy-Trigger:** SSH-Deploy aus GitHub-Hosted-Runner vs. Self-Hosted-Runner direkt auf der VPS.
2. **Public-Exposure:** Klassischer Reverse-Proxy mit offenem Port 443 (Nginx/Caddy + Let's Encrypt) vs. Cloudflare-Tunnel (outbound initiiert, keine offenen Ports).

Alternativen:

Variante	Deploy	Exposure	Bewertung
A	SSH-Action aus GitHub-Hosted	Nginx + Let's Encrypt	klassisch, aber: SSH-Key-Verwaltung, offene Firewall-Regeln (80/443), eigene TLS-Renewal-Logik
B	SSH-Action aus GitHub-Hosted	Cloudflare-Tunnel	weniger Firewall-Aufwand, aber: SSH-Key bleibt Angriffsfläche
C	Self-Hosted-Runner auf VPS	Cloudflare-Tunnel	kein SSH-Pfad, kein offener Port, lokale Docker-Builds nutzen lokalen Cache

Entscheidung: Option C — Self-Hosted-Runner direkt auf der VPS in Kombination mit Cloudflare-Tunnel.

Begründung:

- Der Runner ist gleichzeitig Docker-Host → `docker build` + `docker compose up` laufen ohne Image-Transport (lokaler Daemon-Cache wird zwischen Runs wiederverwendet).
- Kein SSH-Mechanismus nötig → kein Deploy-Key, keine `appleboy/ssh-action`-Dependency.
- Cloudflare-Tunnel ist outbound initiiert → keine eingehenden Ports auf der VPS offen; TLS terminiert bei Cloudflare (managed Cert, automatische Erneuerung).
- DDoS-Mitigation und WAF inkludiert.
- Der GHCR-Push (Tags `latest` + `<short-sha>`) bleibt erhalten — die Production-Compose nutzt zwar das lokal gebaute Image, aber die Tags in der Registry erlauben jederzeit Rollback oder externes Pull-Deployment.

Konsequenz:

- (+) Push auf `main` → automatischer Live-Deploy in unter 2 Minuten inkl. Smoke-Test gegen `https://web17skill.com/health`.
- (+) Keine externe Cloud-Plattform — VPS bleibt unter eigener Kontrolle, Cloudflare ist nur als Tunnel-Endpoint involviert (keine Anwendungslogik bei Cloudflare).

- (+) Container bindet bewusst nur auf Loopback (`127.0.0.1:80:8080`) — selbst bei Cloudflare-Tunnel-Ausfall ist die App nicht von außen direkt erreichbar.
- (–) Der Self-Hosted-Runner ist ein zusätzlicher Service auf der VPS, der gewartet werden muss (Updates, Logs).
- (–) Wenn die VPS offline ist, kann nicht deployed werden (kein Fallback auf GitHub-Hosted-Runner). Für den Submission-Kontext akzeptabel.

Operatives Setup:

- `.github/workflows/deploy.yml` mit `runs-on: self-hosted`
- `docker-compose.prod.yml` mit Loopback-Bind und ohne SQL-Port-Publish
- `cloudflared` als systemd-Service auf der VPS
- GitHub-Secrets: `GHCR_TOKEN` (Classic-PAT), `MSSQL_SA_PASSWORD`

→ Vollständiges Setup in [§7.6 Production-Deployment](#).

Übersicht aller ADRs

ADR	Titel	Bezug
ADR-001	Stack: .NET 10 + Blazor Server + MS-SQL	§7 , §8
ADR-002	Cage relational modelliert	ER-Modell, Datenbank-Skript
ADR-003	Highscore als View	UC08, <code>vw_Highscore</code>
ADR-004	Cell-Uniqueness via Trigger	UC04, V06
ADR-005	AppUser statt User	T-SQL Reserved
ADR-006	RowIdx/ColIdx statt Row/Col	T-SQL Reserved
ADR-007	Keine Solution-Spalte	UC04 strict
ADR-008	Pencil Marks als eigene Tabelle	UC14
ADR-009	Pause via TotalPausedSeconds	UC13
ADR-010	Solver limit 2 Lösungen	UC11, V07
ADR-011	Test-Pyramide	README §3.2
ADR-012	Score-Formel	UC08
ADR-013	Hint-Strategie	UC07
ADR-014	3 zusätzliche UCs	README §2.3
ADR-015	Single-Image Docker (DB + App)	§7.5
ADR-016	Production-Deploy via Self-Hosted Runner + Cloudflare-Tunnel	§7.6

Verweise

- [Kapitel 1 — Einführung und Ziele](#)
- [Kapitel 7 — Verteilungssicht](#)
- [Kapitel 8 — Querschnittliche Konzepte](#)
- **Use-Cases-Dokument**
- **Validation-Regeln**

- **ER-Modell**
- **Funktionalitäts-Matrix**
- **Datenbank-Skript**
- **Aufgabenstellung** — Aufgabenstellung (autoritativ)

10 Qualitätsanforderungen

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Dieses Kapitel präzisiert die in §1.2 genannten Top-Ziele in Form eines Quality-Trees und konkreter, testbarer Quality-Szenarien.

§10.1 Quality-Tree

Der Tree gliedert das übergeordnete Ziel "Qualität" in fünf Attribute. Jedes Blatt ist über ein Szenario in §10.2 operationalisiert.



Zuordnung Top-Ziele → Quality-Tree-Knoten:

Top-Ziel (siehe §1.2)	Tree-Knoten
Q1 Korrektheit des Solvers	KO1, KO2, PE1
Q2 Lösungs-Eindeutigkeit	KO1 (Branch <code>Solutions ≥ 2</code>), siehe V07
Q3 Test-Abdeckung	WA1

§10.2 Quality-Szenarien

Jedes Szenario ist tabellarisch nach dem arc42-Schema **Quelle · Stimulus · Artefakt · Reaktion · Antwortmaß** beschrieben. Spalte "Test-Anker" verlinkt die geplanten Unit-/Integration-/E2E-Tests (siehe **Test-Protokoll**).

Q01 — Solver-Korrektheit (Eindeutigkeit)

Feld	Wert
Quelle	Wettbewerbs-Prüfer / interner Integration-Test
Stimulus	Solver bekommt ein Puzzle mit genau einer Lösung übergeben
Artefakt	<code>ISolverService.Solve(givenValues, cages)</code>
Reaktion	Solver gibt <code>SolveResult { Solutions = 1, Solution = ...}</code> zurück; bei mehrdeutigem Puzzle stoppt er bei 2 Lösungen und liefert <code>Solutions = 2</code>
Antwortmaß	100% der Test-Cases (beide README-Beispiele + Edge-Cases unlösbar / mehrdeutig) liefern korrekten Solutions-Count
README-Beleg	§1 "The solution must be unique ." + §2.1 UC11 "Solutions must be calculated with an algorithm ."
Test-Anker	UC11 → AC11.1 (siehe use-cases.md UC11)

Q02 — Solver-Performance

Feld	Wert
Quelle	Wettbewerbs-Prüfer (Demo-Lauf)
Stimulus	Solver wird mit schwerem Puzzle (Difficulty 3) aufgerufen
Artefakt	<code>ISolverService.Solve</code> (Backtracking + Constraint-Propagation)
Reaktion	Solver terminiert und liefert Ergebnis
Antwortmaß	≤ 2 Sekunden auf lokaler Test-Maschine
README-Beleg	implizit aus UC05 ("solvability check vor Speichern") und UC07 ("ask for a hint") — bei längerer Laufzeit ist UX nicht akzeptabel
Test-Anker	UC11 → AC11.2

Q03 — Login-Security (kein Username-Leak)

Feld	Wert
Quelle	Externer Angreifer / Security-Reviewer
Stimulus	Login-POST mit unbekanntem Username oder falschem Passwort
Artefakt	<code>Login.razor</code> mit <code>SignInManager<AppUser>.PasswordSignInAsync</code>
Reaktion	Server antwortet mit generischer Fehlermeldung "Username oder Passwort falsch." — keine Unterscheidung zwischen "User existiert nicht" und "Passwort falsch"
Antwortmaß	Response-Body, Statuscode und Antwortzeit identisch für beide Fälle (kein Timing-Leak via getrennten Code-Pfaden)
README-Beleg	§2.1 UC2 "a login is required ." in Kombination mit Sicherheits-Best-Practices (V03)
Test-Anker	UC03 → AC03.1

Q04 — Page-Load (Play-Screen)

Feld	Wert
Quelle	End-Nutzer
Stimulus	Klick auf Puzzle in Liste S4 → Navigation zu Screen S6 (<code>/puzzles/{id}/play</code>)
Artefakt	Blazor-Server-Page <code>PlayPuzzle.razor</code> + initiale SignalR-Verbindung + DB-Query Cages
Reaktion	Grid + Cages + Toolbar werden vollständig gerendert
Antwortmaß	< 1 Sekunde bei MS-SQL Server lokal (Loopback) und Cold-Cache
README-Beleg	impliziert durch §1.1 "appropriate navigation" + "controls for all functions"
Test-Anker	Manueller UI-Test im Test-Protokoll

Q05 — Auth-Cookie-Sicherheit

Feld	Wert
Quelle	Security-Reviewer / Browser DevTools
Stimulus	Erfolgreicher Login (UC03)
Artefakt	ASP.NET Core Authentication Cookie
Reaktion	Cookie wird mit Flags HttpOnly + Secure + SameSite=Lax ausgestellt
Antwortmaß	Set-Cookie-Header enthält alle drei Attribute (binary check: pass/fail)
README-Beleg	§2.1 UC2 "login is required" + Sicherheits-Defaults nach V15
Test-Anker	UC03 → AC03.3

Q06 — Test-Coverage

Feld	Wert
Quelle	Wettbewerbs-Prüfer (Submission-Bewertung)
Stimulus	Build + Test-Lauf via <code>dotnet test</code>
Artefakt	xUnit Test-Suite + Coverlet Coverage-Report
Reaktion	Report zeigt Coverage-Wert pro Projekt
Antwortmaß	≥ 80% Line- + Branch-Coverage über Solver / Service-Layer
README-Beleg	§3.1 "For each use case, create at least one positive, one negative, and, where possible, one test case for boundary conditions." + §3.2 "Running the test cases should be implemented with a test framework. "
Test-Anker	Coverlet-Report als Submission-Artefakt; siehe Top-Ziel Q3 in §1.2

§10.3 Querverweise

- [Top-Ziele §1.2](#) — drei priorisierte Ziele
- Validation V01–V16 — Detailregeln pro Layer (Client / Server / DB)
- Use Cases UC01–UC14 — Acceptance Criteria (AC) je UC
- **Test-Protokoll** — konkrete Test-Cases mit Erwartung
- [Kapitel 11 — Risiken](#) — Risiken, die die Qualitätsziele bedrohen

11 Risiken und technische Schulden

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Dieses Kapitel listet Wettkampf-spezifische Risiken (R01–R08) sowie bewusste technische Schulden (TD01–TD03) zum Stand der Architektur-Festschreibung. Bewertungs-Skalen:

- **Wahrscheinlichkeit (W):** niedrig / mittel / hoch
- **Auswirkung (A):** niedrig / mittel / hoch / kritisch (kritisch = führt direkt zu UC-/AC-Verletzung)

§11.1 Wettkampf-Risiken

R01 — Solver-Performance bei worst-case-Cage-Konfigurationen

Attribut	Wert
Beschreibung	Backtracking-Solver kann bei dünn besetzten oder ungünstig zerlegten Cage-Layouts in eine kombinatorische Explosion laufen.
Wahrscheinlichkeit	mittel
Auswirkung	hoch — Verletzung von Quality-Szenario Q02 (≤ 2 s)
Mitigation	Constraint-Propagation pro Cage (Min-/Max-Sum-Bounds), Variable-Ordering MRV (Most-Restricted-Variable), harter Timeout im Service.
Test-Anker	UC11 → AC11.2 (siehe use-cases.md UC11)

R02 — Multi-Solution-Erkennung übersehen (Pattern 2 "Spec-Detail-Blindheit")

Attribut	Wert
Beschreibung	Naïve Solver-Implementierung gibt die erste gefundene Lösung zurück und stoppt — verletzt das README-Strict-Wort "The solution must be unique. " (§1). Multi-Solution-Puzzles würden fälschlich gespeichert.
Wahrscheinlichkeit	hoch (klassischer Fehler bei Killer-Sudoku-Implementierungen)
Auswirkung	kritisch — Verletzung von V07 und Top-Ziel Q2
Mitigation	Solver sucht aktiv nach 2. Lösung und bricht erst dann ab; <code>SolveResult.Solutions ∈ {0, 1, 2}</code> ; UC05 speichert ausschliesslich bei <code>Solutions == 1</code> .
Test-Anker	UC05 → AC05.2; UC11 → AC11.1

R03 — SQL-Constraint-Lücken (Cage-Trigger fehleranfällig)

Attribut	Wert
Beschreibung	Der INSTEAD-/AFTER-Trigger <code>trg_CageCell_UniquePerPuzzle</code> (siehe Datenbank-Skript ab Zeile 99) ist die einzige DB-seitige Garantie, dass eine Zelle nicht in zwei Cages desselben Puzzles liegt. Fehler im Trigger oder Batch-Inserts mit deaktiviertem Trigger könnten inkonsistente Daten zulassen.
Wahrscheinlichkeit	niedrig
Auswirkung	hoch — Datenintegritäts-Verletzung gegenüber V06
Mitigation	Defense in Depth — Application-Layer-Validierung in <code>IPuzzleService.ValidateStructureAsync</code> zusätzlich zur Trigger-Prüfung; Integration-Test mit absichtlich kollidierendem Insert.
Test-Anker	UC04 → AC04.2; Integration-Test "Doppelte Cage-Cell wird vom Trigger geblockt"

R04 — Blazor-Server-Circuit-Disconnects bei langen Pausen (UC13)

Attribut	Wert
Beschreibung	Blazor-Server-Apps halten pro User eine SignalR-Circuit (siehe Funktionalitäts-Matrix "SignalR-Verbindung permanent"). Bei Pause + Tab-Wechsel kann die Circuit getrennt werden — laufende Game-State-Manipulationen würden verloren gehen, falls UC13-Auto-Save nicht greift.
Wahrscheinlichkeit	mittel
Auswirkung	mittel — UC13 AC13.2 (Resume restored alle Zellen) bedroht
Mitigation	Persistenz pro Move (UC13 "Browser-Tab schliessen ohne Pause → State wird trotzdem persistiert (auto-save bei jedem Move)"), serverseitiges Resume aus DB statt aus Circuit-State.
Test-Anker	UC13 → AC13.1, AC13.2

R05 — Test-Coverage-Schwächen bei Solver-Edge-Cases

Attribut	Wert
Beschreibung	Solver hat viele Code-Pfade (Cage mit 1 Zelle, Cage über ganze Zeile, unlösbares Puzzle, mehrdeutiges Puzzle, leeres Grid). Ohne gezielten Edge-Case-Plan kann die 80%-Marke (Quality-Szenario Q06) am Solver allein scheitern.
Wahrscheinlichkeit	mittel
Auswirkung	hoch — verletzt Top-Ziel Q3 und damit Submissions-Qualität
Mitigation	Solver-Unit-Tests systematisch nach der Edge-Case-Liste in <code>functionality.md</code> "Kritische Abhängigkeiten" (5+ Tests: README-Beispiele, unlösbar, mehrdeutig, 1-Zell-Cage, voller Zeilen-Cage).
Test-Anker	UC11 — separate Test-Klasse <code>SolverServiceTests</code>

R06 — Sum-Check Pre-Validation übersehen (Pattern 5 "Improvisierter Format-String")

Attribut	Wert
Beschreibung	README §2.3 verlangt explizit: "Use the value [Total = 405] for a 'simple' validation before checking the solution with an algorithm." Eine Implementierung, die direkt den vollen Algorithmus aufruft, ignoriert die README-Vorgabe — auch wenn das Endergebnis korrekt ist.
Wahrscheinlichkeit	mittel
Auswirkung	mittel — Verletzung einer explizit benannten Spec-Anforderung
Mitigation	<code>IGameService.CheckSolutionAsync</code> ruft als ersten Schritt den Sum-Check (siehe V09) auf; Unit-Test verifiziert, dass bei <code>Summe ≠ 405</code> der Solver nicht aufgerufen wird.
Test-Anker	UC09 → AC09.1

R07 — Spec-Verletzung "No solution is recorded" durch versehentliches Speichern

Attribut	Wert
Beschreibung	README §2.1 UC4: "No solution is recorded." Wenn z.B. zu Debug-Zwecken oder per ORM-Auto-Migration eine Spalte <code>Solution</code> in <code>Puzzle</code> landet, wird die Spec verletzt — auch wenn die Spalte leer bleibt.
Wahrscheinlichkeit	niedrig
Auswirkung	hoch — direkte Verletzung eines Strict-Wortes der Spec
Mitigation	Schema in Datenbank-Skript ohne Solution-Spalte; Integration-Test gegen <code>INFORMATION_SCHEMA.COLUMNS</code> stellt sicher, dass keine <code>Solution</code> -Spalte existiert (siehe ER-Modell).
Test-Anker	UC04 → AC04.4

R08 — Reserved-Keyword-Konflikte in T-SQL (User, Row, Col)

Attribut	Wert
Beschreibung	Die in der Domäne natürlichen Namen <code>User</code> , <code>Row</code> , <code>Col</code> sind in T-SQL reservierte Keywords. Ohne Anpassung schlagen DDL-/DML-Statements fehl oder benötigen Bracket-Quoting <code>[User]</code> .
Wahrscheinlichkeit	hoch (würde ohne Umbenennung sicher auftreten)
Auswirkung	mittel (Build-Fail, leicht zu beheben — aber unschön und Quelle für Folgebugs)
Mitigation	Umbenennung gemäss erm.md Design-Entscheidungen: Tabelle <code>AppUser</code> statt <code>User</code> , Spalten <code>RowIdx</code> / <code>ColIdx</code> statt <code>Row</code> / <code>Col</code> . Bereits konsistent in Datenbank-Skript umgesetzt.
Test-Anker	Build-/Migrations-Lauf des Datenbank-Skript -Scripts

§11.2 Technische Schulden (Stand 2026-05-27, vor App-Implementierung)

Die folgenden Schulden sind **bewusste Entscheidungen** für den Wettkampf-Scope. Jede Schuld nennt den Grund und die Konsequenz für einen späteren Produktiv-Einsatz.

TD01 — Doppelte Schema-Wahrheit: EF-Migrations + generiertes sudoku.sql

Attribut	Wert
Beschreibung	EF-Migrations sind Source of Truth (<code>KillerSudoku.Data/Persistence/Migrations/</code>). Das von README §1.3 geforderte Datenbank-Skript wird per <code>dotnet ef migrations script -- idempotent -o db/sudoku.sql</code> aus den Migrations exportiert.
Begründung	README §1.3 verlangt explizit ein Datenbank-Skript für den Prüfer; gleichzeitig braucht ein .NET-/EF-Workflow Migrations für reproduzierbare Schema-Evolution. Beides parallel zu pflegen ist günstiger als nur eine Variante.
Folge bei Produktiv-Use	Bei manuellen Änderungen am SQL-Skript ohne entsprechende Migration entsteht Drift — Disziplin nötig: Schema-Änderungen immer über Migrations, dann Datenbank-Skript neu exportieren. Im CI kann ein Drift-Check ergänzt werden (<code>dotnet ef migrations script</code> ↔ committed Datenbank-Skript vergleichen).

TD02 — Keine i18n-Vorbereitung (UI nur Deutsch)

Attribut	Wert
Beschreibung	UI-Texte (Fehlermeldungen, Labels, Validierungs-Strings) sind direkt in Razor-Components hardcoded in Deutsch . Kein <code>IStringLocalizer</code> , keine <code>.resx</code> -Files.
Begründung	Wettkampf-Scope; README definiert keine Mehrsprachigkeit; reduziert Aufwand zugunsten Test-Coverage.
Folge bei Produktiv-Use	Vollständiges Refactoring zu Localizer notwendig für Englisch/Französisch; Test-Updates für lokalisierte Strings.

TD03 — Keine Multi-User-Concurrency-Tests (Blazor-Server-Single-Circuit-Annahme)

Attribut	Wert
Beschreibung	Tests gehen davon aus, dass pro User maximal eine aktive Blazor-Circuit existiert. Race-Conditions bei parallelen Tabs / mehreren Devices desselben Users werden nicht durch Tests abgedeckt.
Begründung	Wettkampf-Scope. UC13 AC13.3 fordert lediglich "Nur 1 aktives Game pro User-Puzzle-Kombi gleichzeitig" — das ist über den DB-Filtered-Index <code>UX_Game_ActiveOnly</code> (siehe Datenbank-Skript Zeile 155) abgesichert, nicht über App-Tests.
Folge bei Produktiv-Use	Concurrency-/Last-Tests erforderlich; ggf. Optimistic-Concurrency-Token auf <code>Game</code> (rowversion).

§11.3 Querverweise

- [§10.2 Quality-Szenarien](#) — die durch Risiken bedrohten Antwortmasse
- Validation V01–V16
- Use Cases mit Acceptance Criteria
- ER-Modell und Design-Entscheidungen
- DB-Schema

12 Glossar

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument
(autoritativ): **Aufgabenstellung**

Alle fachlichen und technischen Begriffe der Architektur-Doku, alphabetisch sortiert.
Quellenangaben verweisen auf das Wettbewerbs-README (**Aufgabenstellung**) bzw. die
ergänzenden Quell-Dokumente unter **docs/**.

Term	Definition · Quelle
AC (Acceptance Criterion)	Testbares Kriterium pro Use Case; mappt 1:1 auf einen Test-Case im Test-Protokoll. Siehe Notation in Use-Cases-Dokument .
AppUser	Benutzer-Entität in der DB. Namens-Workaround, weil <code>User</code> ein reserviertes T-SQL-Key-Word ist (siehe R08). Tabellen-Definition: Datenbank-Skript Zeile 36.
arc42	Architektur-Dokumentations-Template (arc42.org), v8.2, 12 Kapitel. Siehe Index 00-README.md .
Auto-Solver	Algorithmus aus UC11, der ein Puzzle automatisch löst. Service-Interface <code>ISolverService</code> . Siehe use-cases.md UC11.
Backtracking	Solver-Strategie: rekursive Tiefensuche mit Constraint-Checks, Rücksprung bei Konflikt. Eingesetzt im <code>SolverService</code> für Killer-Sudoku (UC11).
bUnit	.NET Testing Library für Blazor-Component-Tests. Verwendet für Component-Level-Tests von <code>PuzzleGrid.razor</code> etc. (siehe Funktionalitäts-Matrix).
Cage	Zell-Gruppe mit vorgegebener Summe (Killer-Sudoku-spezifisch). README §1: "The sum of all numbers in a cage must match the small number printed in its corner."
Cage-Sum	Soll-Summe eines Cages, Wertebereich 1–45 (1 Zelle min 1, 9 Zellen max $1+...+9 = 45$). DB-Constraint <code>CK_Cage_Sum_Range</code> in Datenbank-Skript .
Cell / Zelle	Ein Feld im 9×9-Grid; identifiziert durch <code>(RowIndex, ColIdx)</code> mit Werten 0–8. Wert einer Spielzelle: 1–9 oder leer.
Difficulty	Schwierigkeitsgrad 1, 2 oder 3. README §2.1 UC4: "the difficulty level from 1-3". DB-Constraint <code>CK_Puzzle_Difficulty</code> .
Game	Eine Spielsession eines Users an einem Puzzle. Tabellen-Definition: Datenbank-Skript Zeile 128.
Hint	Algorithmischer Tipp; UC07. Strategie: Naked-Single → Cage-Forced → Solver-Fallback (siehe use-cases.md UC07).
Highscore	Score-Ranking gemäss Formel UC08; persistiert als View <code>vw_Highscore</code> (siehe Datenbank-Skript Zeile 215).
HintsUsed	Anzahl der in einem Game verwendeten Hints; score-relevant gemäss UC08-Formel. README §2.1 UC8: "based on the time needed and the number of hints used".
Killer Sudoku	Sudoku-Variante mit zusätzlichen Cage-Summen. README §1: "various tricky variants. One variant is to fill in the squares based on a set of sums over several squares."
Nonet	Ein 3×3-Block im Sudoku-Grid; insgesamt 9 Nonets. README §1: "Each row, column, and nonet contains each number exactly once".
Pencil Mark	Kandidaten-Notiz pro Zelle (kleine Markierungen 1–9 als mögliche Werte). UC14, eigene Tabelle <code>PencilMark</code> .

Puzzle	Eine Killer-Sudoku-Aufgabe (Cages + Difficulty). README §2.1 UC4: "No solution is recorded" — Lösung wird nicht persistiert.
Score	Punkte gemäss Formel <code>max(0, 10000 - TimeSeconds - HintsUsed × 300)</code> (siehe use-cases.md UC08).
SignalR	Real-Time-Kommunikations-Framework für Blazor-Server-Circuits. Permanente Verbindung Server ↔ Client (siehe functionality.md "Stack-Konventionen").
Solvability	Existenz mindestens einer Lösung. UC05-Bedingung: "The puzzle can only be saved if it is solvable." (README §2.1 UC5).
Solver	Domänen-Service <code>ISolverService</code> (UC11). Interface-Definition: functionality.md "Service-Interfaces".
Sum-Check 405	Schnell-Validierung: Summe aller 81 Zellen-Werte = $9 \times 45 = 405$. README §2.3: "Calculate the sum of all numbers in the completed Sudoku. The value can be determined unambiguously." Eingesetzt vor der vollständigen Algorithmus-Validierung. Siehe V09 .
TDD	Test-Driven Development; Workflow Red → Green → Refactor. README §3.1: "Plan your test cases before you start the implementation."
TimeSeconds	Spieldauer in Sekunden, abzüglich Pausen: <code>TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds</code> . Score-relevant (UC08). Siehe UC10 und UC13 .
Uniqueness	Lösung muss eindeutig sein. README §1: "The solution must be unique ." Strict-Wort; Multi-Solution-Puzzles werden abgelehnt (siehe R02 , V07).
Use Case	Anwendungsfall, Notation siehe Use-Cases-Dokument . UC01–UC11 aus README §2.1, UC12–UC14 selbst gewählt gemäss README §2.3.
V01–V16	Validation-Regeln aus Validation-Regeln . Jede Regel ist Layer-zugeordnet (Client / Server / DB).
xUnit	.NET Unit-Testing-Framework, Standard-Wahl für Service-/Solver-Tests (siehe functionality.md "Kritische Abhängigkeiten").